

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 16:58:03
Файлов исходного кода: 87
Суммарный объём кода: 1.18 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/DfsBridgesSimulator.tsx
37. src/components/DijkstraViz.tsx
38. src/components/DPVisualizer.tsx
39. src/components/EulerSimulator.tsx
40. src/components/ExpressQuiz.tsx
41. src/components/FloydViz.tsx
42. src/components/GraphTraversalViz.tsx
43. src/components/HeapViz.tsx
44. src/components/hints/demoKit.tsx
45. src/components/hints/demosExtra.tsx
46. src/components/hints/demosGraph.tsx
47. src/components/hints/demosStruct.tsx
48. src/components/hints/MiniDemo.tsx
49. src/components/hints/SimulatorModal.tsx
50. src/components/hints/TermPopover.tsx
51. src/components/JohnsonViz.tsx
52. src/components/KosarajuViz.tsx
53. src/components/KruskalSimulator.tsx
54. src/components/MnemonicCards.tsx
55. src/components/MobileToc.tsx
56. src/components/Navbar.tsx
57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```

Универсальное пособие • полный исходный код

```
-----
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ,,,
+

## `github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml
name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (
|
→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "*"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
```

```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
На Ubuntu политика ImageMagick иногда запре
разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.densi  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages  
 echo "| \`public/export/full\_code\_all\_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB\_STEP\_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```

 <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

```
`public/favicon.svg`
```

```
Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
</svg>  
....
```

```
### Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
....
```

```
`src/App.tsx`
```

```
Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from  
import { chapters } from "../data/content";
```



```

    }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />

          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
            {/* Сайдбар только на десктопе – на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0">
              <Sidebar selectedChapterId={activeChapterId}>
            </div>

            <main id="main-content" className="flex-1 min-w-0">
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between">
                <div className="min-w-0">
                  <div className="text-[10px] uppercase">
                    {activeChapter.category || "Универсальное"}
                  </div>
                  <h2 className="text-base sm:text-xl font-medium">
                    {activeChapter.title}
                  </h2>
                </div>
                <ChapterNav prev={prev} next={next} index={index}>
              </div>
            </main>
          </div>
        </>
      ) : (
        <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
          <div className="hidden lg:block lg:w-80 shrink-0">
            <Sidebar selectedChapterId={activeChapterId}>
          </div>

          <main id="main-content" className="flex-1 min-w-0">
            {/* Шапка главы с быстрыми переходами */}
            <div className="flex items-center justify-between">
              <div className="min-w-0">
                <div className="text-[10px] uppercase">
                  {activeChapter.category || "Универсальное"}
                </div>
                <h2 className="text-base sm:text-xl font-medium">
                  {activeChapter.title}
                </h2>
              </div>
              <ChapterNav prev={prev} next={next} index={index}>
            </div>
          </main>
        </div>
      )}
    </Navbar>
  </div>
);

```

```

        </footer>
    </div>
  );
}
....

### Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
 </>
 </main>
) : (
 <main>
 <ChapterNav />
 <ChapterView />
 </main>
)
}

```

```

<div className="flex items-center gap-3 w-full" >
 <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1" >
 <h1 className="text-xl font-extrabold text-slate-900" >
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium" >
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <CodeIcon className="w-4 h-4" /> Симулятор
 </button>
</div>

```

```


 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

```

Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.asm.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
 }
 return runtimePromise;
}
```

```

 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.end`
 setRuntimeMessage("Не удалось выполнить код – про
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>
 </div>
 </div>
 </main>

```

```

 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] res
-none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

<div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
eholder:text-slate-700"
 />
</div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"}

 </div>
 <pre className="min-h-[360px] max-h-[560px]
x] leading-6 text-slate-300">
 {output}
 </pre>

```

Универсальное пособие • полный исходный код

```

+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`
+
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) => void }) => void
+ setStderr: (options: { batched: (message: string) => void }) => void
+ setStdin: (options: { stdin: () => string | number | null }) => void
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
```



```

+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу.
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1.
+ <section className="rounded-2xl border borde
+ <div className="flex flex-wrap items-cente
+ <div className="flex items-center gap-2"
+ <span className="w-2.5 h-2.5 rounded-f
+ <span className="text-sm font-bold tex
+ <span className="text-[11px] text-slate

```

```

+
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #334155; border-top: none; margin-top: 10px; padding: 10px 10px 0 10px;}}
+ <label className="block px-4 pt-3 text-slate-600" style={{display: block; margin-bottom: 5px; padding-left: 10px;}}
+ Ввод для input() • по одной строке на строку
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tran
+ style={{border: 1px solid #334155; border-top: none; margin-top: 10px; padding: 10px 10px 0 10px;}}
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #334155; border-radius: 12px; margin-top: 10px; padding: 10px 10px 0 10px;}}
+ <div className="flex items-center justify-between" style={{border: 1px solid #334155; border-radius: 12px; margin-bottom: 10px; padding: 5px 10px;}}
+ <div className="flex items-center gap-2" style={{flex: 1;}}
+ <Terminal className="w-4 h-4 text-emerald-500" style={{border: 1px solid #334155; border-radius: 12px; margin-right: 10px;}}
+ </div>
+ <span className={`inline-flex items-center gap-2`} style={{font-family: monospace; font-size: 0.9em; color: #334155;}}
+ {runtimeReady && <CheckCircle2 className="h-4 w-4 text-emerald-500" style={{height: 1em; width: 1em;}}
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #334155; border-radius: 12px; margin-top: 10px; padding: 10px 10px 0 10px;}}
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4" style={{border: 1px solid #334155; border-top: none; margin-top: 10px; padding: 10px 10px 0 10px;}}
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" style={{margin-top: 10px;}}
+ <div className="flex gap-2 rounded-xl border

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
},
server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
},
preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
},
});
```
```

Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
```

Универсальное пособие • полный исходный код

- 
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
  3. **\*\*Строгая структура контента:\*\***
    - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
    - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
    - Обязательная сетка аналогий (2 колонки: неформальный и формальный)
    - "Душные" детали (код, формулы, сложные доказательства)
  4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

---

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

---

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "version": "1.0.0",
 "description": "A guide to the ExamPrep Reader, a tool for preparing for the ExamPrep exam.",
 "keywords": "ExamPrep, Reader, Guide, ExamPrep Reader, ExamPrep Reader Guide",
 "author": "Remix Remix",
 "license": "MIT",
 "repository": {
 "type": "git",
 "url": "https://github.com/remix-remix/exam-prep-reader"
 },
 "scripts": {
 "dev": "vite dev",
 "build": "vite build",
 "preview": "vite preview"
 },
 "dependencies": {
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-router-dom": "^6.14.2",
 "vite": "^4.3.9"
 },
 "devDependencies": {
 "@types/react": "^18.2.14",
 "@types/react-dom": "^18.2.6",
 "@types/react-router-dom": "^6.14.2",
 "typescript": "^5.0.4"
 }
}
```

```
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

---

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5174
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- \* **\*\*Содержание\*\*** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симуля
- Словарь – `src/data/glossary.ts`, мини-анимации – `src
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модальки из подсказки.

## ## Структура

...

- src/
  - App.tsx – оболочка, привязка глав к визуалу
  - components/ – интерактивные симуляторы (Дейкстра, сортировка)
  - data/ – контент пособия (главы/билеты)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновление)
- .github/workflows/ – автоматизация

— — —

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен  
азуемые теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах ``<r>``.

Универсальное пособие • полный исходный код

```

> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет         | Тема                           | Статус                                  |
|---------------|--------------------------------|-----------------------------------------|
| ---           | ---                            | ---                                     |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии. |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть блок аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть блок аналогии. |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динамическое программирование\*\***

|    |                                               |   |   |
|----|-----------------------------------------------|---|---|
| 9  | 9. Графы. Топологическая сортировка           | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | 1 | - |
| 12 | 12. Графы. Точки сочленения                   | 1 | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 | - |
| 16 | 16. Графы. Кратчайший путь. Флойд             | - | - |
| 17 | 17. Графы. Алгоритм Джонсона                  | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 | - |
| 21 | 21. Префикс-функция (π), КМП                  | 4 | - |
| 22 | 22. Z-функция                                 | 4 | - |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 | - |
| 24 | 24. Классы сложности, сведение задач          | 4 | - |
| -  | Обзор: Кратчайшие пути и Графы                | - | - |
| -  | Алгоритмы на карте                            | - | - |
| -  | Эйлер: Путь ≠ Цикл                            | - | - |
| -  | Мосты в графах: код + визуализация            | 1 | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inline`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
```



```

import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz)?.scrollIntoView();
 });
 });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
 {activeTab === "guide" ? (
 <div>
 <MobileToc
 index={index}
 chapter={chapter}
 progress={progress}
 next={next}
 prev={prev}
 goTo={goTo}
 compilerOpen={compilerOpen}
 onToggleCompiler={onToggleCompiler}
 />
 </div>
) : (
 <div>
 <MobileCode
 index={index}
 chapter={chapter}
 progress={progress}
 next={next}
 prev={prev}
 goTo={goTo}
 compilerOpen={compilerOpen}
 onToggleCompiler={onToggleCompiler}
 />
 </div>
)}
 </div>
 </div>
);

```

```

 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>
) : (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
)}

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное руководство
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

{compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>

```

```

* , чтобы на
* повесить всплывающую карточку (как превью ссылок в В
*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 }
 }
}
```

```
 try {
 annotateTerms(el);
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 KB

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
```

```

 from {
 opacity: 0;
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
 }

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

```

```
.chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
}
.chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/87: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

---



## Аналогия 2: Ступени

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приори
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`,
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <!-- 0. Определение -->
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-lg sm:text-xl font-mono">
 поиска без
 : серия поворотов, которая поднимает v в кор
 <p class="text-sm text-slate-400 text-c">
 т – поворот.</p>
 </div>

```

```

<ul class="list-disc list-inside text-slate-300">
 Важно!
 каждой вставки чинят дерево, чтобы оно не

 Важно!
 менно быть кривым. Но каждый раз, когда мы
 ровнее становится.

</div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 Аналогия 1: стопка бумаг на столе
 </div>
 <p class="text-slate-300 text-sm">У тебя
 скиваешь его и, поработав, кладёшь Важно!
 неделю самые нужные бумаги сами собой оказыва
 ет ровно это: «взял → положил наверх».</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
 Аналогия 2: тропинка через газон
 </div>
 <p class="text-slate-300 text-sm">Ты идёшь по дорожке, ко
 ода дорожка сама укорачивалась вдвое: чем чаще
 оплачивает все следующие. Пойдёшь один раз
 </p>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-4">Поворот
 <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки и

```

```

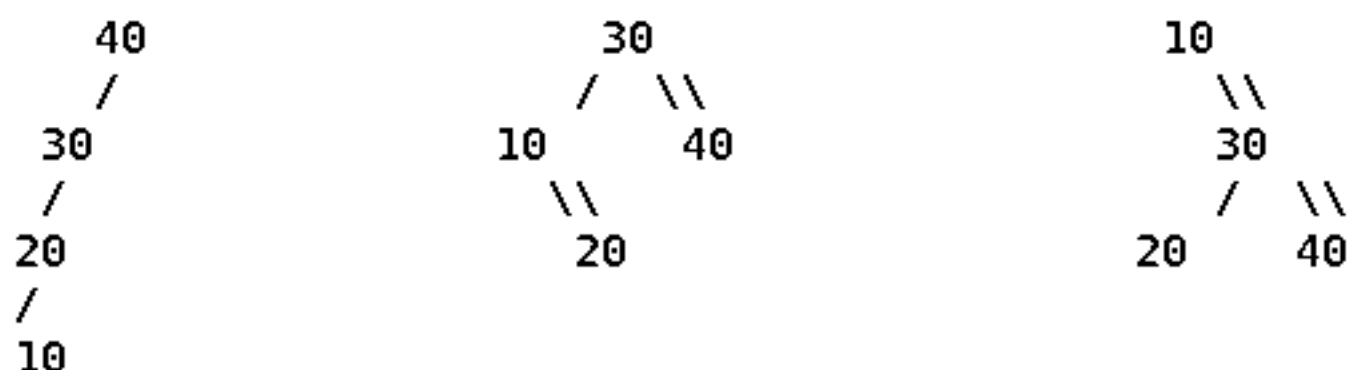
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">Зиг</td>
 <td class="py-3 pr-3">деда</td>
 <td class="py-3 pr-3">сам х</td>
 <td class="py-3">х стал королем</td>
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">Пег</td>
 <td class="py-3 pr-3">х и п</td>
 <td class="py-3 pr-3">аво-право)</td>
 <td class="py-3">х поднялся</td>
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bolder">Григ</td>
 <td class="py-3 pr-3">х и п</td>
 <td class="py-3 pr-3">Пег</td>
 <td class="py-3">п и г стали</td>
 </tr>
 </tbody>
</table>
</div>

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

<!-- 5. Главная ловушка -->
<div class="bg-rose-950/30 p-6 rounded-xl border">
 <h3 class="text-lg font-bold text-rose-300">
 <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
 <div class="grid grid-cols-1 lg:grid-cols-2">

```

```
<p class="text-slate-300 text-sm mb-4">Дерево
3 шага, а потом делаем Splay(10).</p>
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-
0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```



итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2
<h3 class="text-lg font-bold text-white mb-3">
<p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag
ах идея такая:</p>
<div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">H
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">д
```

```

// один поворот: x встаёт на место своего родителя
function rotate(x) {
 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда нет
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
 }
 return x; // теперь x —
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-
 <summary class="p-4 font-bold text-slate-400
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
 </summary>
 <div class="p-5 text-sm text-slate-300 space-
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Последний
 енный или преемник). Splay делают всегда, да
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Treap дер
 play не хранит вообще ничего и держит балан
 p>
 </div>
 </div>
</details>

</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 }

```

```

 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white"
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

 description: "Разбиение орграфа на макро-вершины. А
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 БЫХ направлениях по односторонним улицам. Как это отразится
 у на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутые",
 content: `

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-rose-300 italic"
 <p class="text-xl font-mono text-white"
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

```

```

 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-6">
изкому племени для объединения. К вечеру пл

```

```

 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP —
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведен
 аче B, то B — это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

ин раз. Вершины повторять можно.</p>

```
<p><strong class="text-white">Эйлер
<div class="p-3 bg-slate-950 rounded-lg border-rose-500 shadow-md">
 <p class="text-sm text-blue-300">
 <p><strong class="text-white">Путь:
 <p><strong class="text-white">Цикл:
 </div>
</div>
</div>
```

```
<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла)

• шел-вошёл-вышел).

• финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-950 border-rose-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная).
 Теплый пришёл в бар (вторая нечётная).
 Домой вернуться не смог, потому что
 </p>
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь)
 </div>
 <p class="text-slate-300 text-sm mb-4">
```

```
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Мосты
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Вот тебе задание. Ты альпинист. Ты спустился в пещеру.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold text-white">Мост: Единственная веревка
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <strong class="text-white">
 Единственная ниточка. Порвётся — граф р
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только одна ниточка.
 Если верёвка (ребро v-u) оборвётся — граф разойдётся.
 </p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только одна ниточка.
 Если верёвка (ребро v-u) оборвётся — граф разойдётся.
 </p>
 </div>
</div>
```



```

int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `<code class="text-white">V=5, -white">10 &gt; 9</code>` – БАХ, непланарен!

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забу

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `<code class="text-white">V=6 xt-white">9 ≤ 12</code>` – проходит, но он в

Почему? Потому что у двудольного гр

Отсюда более строгое ограничение: `<`

`<code class="text-white">9 &gt; 8</code>`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скры

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 spac
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен.  $V = 5$ ,  $E = 10$ .

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекроют (удалят вершину) – районы будут
 хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит :
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень) */
 func: string;
 /** Локальные переменные: имя → герг (усечённый). */
 locals: Record<string, string>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Текст последней необработанной ошибки (или пустая) */
 error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
 return `import sys as __sys, json as __json`
```

```
`;
}
```

```
/**
```

```
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как amber-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
```

```
export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
): Record<string, boolean> {
 const out: Record<string, boolean> = {};
 for (const k of Object.keys(cur)) {
 out[k] = !prev || !(k in prev) || prev[k] !== cur[k]
 }
 return out;
}
```

```
/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name
```

```
/**
```

```
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
```

```
export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
}
```

```

 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину —
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS — обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся — откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре
 demo: "heap",
 simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь свер
 formal: "push / pop / top за O(1). На стеке живёт р
 demo: "stack",

```

```

 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то обход»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксную",
 "о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень",
 "кладём с в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла",
 "строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 "циклов.",

```



```

 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь массив, а его дети — за половинные куски.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается на две части.",
 formal: "Ребро (u, v) — мост, если $low[v] > tin[u]$, где $tin[u]$ — время захода в вершину u при обходе в глубину, а $low[v]$ — минимальное время захода в вершину v или её потомков (если ребро в родителя — нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точки разрыва"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: увольь её — и коллапсирует половина компании.",
 formal: "Для не-корня: v — точка сочленения, если существует ребро (u, v) такое, что $low[v] > tin[u]$, где $tin[u]$ — время захода в вершину u при обходе в глубину, а $low[v]$ — минимальное время захода в вершину v или её потомков (если ребро в родителя — нет).",
 demo: "articulation",
 simulator: "graph-articulation",
},

```

```

 demo: "np",
 },
 {
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»:
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h – ра
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
 },
 {
 id: "scc",
 labels: ["компонента сильной связности", "компоненты
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до кажд
 formal: "Косарайю: DFS с сохранением порядка выхода
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
 },
 {
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу
 "префикс", "префикса", "суффиксом", "суффикс", "с
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начал
 от кусок.",
 formal:
 " $\pi[i]$ – длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 }

```

```

 formal:
 "Дейкстра — неотрицательные веса, $O(E \log V)$. Форд-Беллман, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
 },
 {
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикла"],
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»",
 formal:
 "Форд-Беллман: если на V -й итерации ещё удаётся с",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "самое дешёвое ребро",
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разрезе»",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:

```

```
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс"],
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший к
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали кучу,
 complexity: "O(E log V) с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман"],
 title: "Алгоритм Форда-Беллмана",
 short:
 "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам,
 ",
 formal:
 "После k-й итерации гарантированно верны все кратчайшие
 ось — есть отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки
 шить пересадку в k?",
 formal:
```

```

 "Сортировка $O(E \log E)$ + E операций union/find. C
complexity: "O(E log E)",
demo: "mst",
simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Горо
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица
demo: "graph",
simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с

```

```

 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больш
 ый отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведён
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы O(1). Минимум без изменен
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с

```

```

 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) :

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string;
 e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);

```

-----  
 ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

-----  
 import { Chapter } from "../types";

```

export const graphChapters: Chapter[] = [

```

```

 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Позитивное плани
 </div>
 <p class="text-slate-300 text-sm mb
(нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstr
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb
охождение улицы), Дейкстра сломается, так к
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную
g V). Гарантирует корректность только для н
 </div>
 </details>
```



```

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-1">
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg

```

```
<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Тр
</div>
<p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мос
 <p>
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 outl
 pen:border-slate-700/50">
```

```

 q.push(child);
 }
 }
 }
 </div>
</div>
</details>
</div>
</section>`
 },
 {
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Алгори
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
 </div>
 </section>`
 }
];

```

```

 "Эйлера пути нет – нужно удалять лишние рёбра
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерово
 ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В нашем
 },
 {
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа в v
 поиска мостов и точек сочленения."
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов",
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро"
 }
]

```

ФАЙЛ 24/87: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 48 | РАЗМЕР: 1.5 КБ

```
import { createContext, useContext, useEffect } from "react";

/**
 * Связь «компилятор ↔ открытая визуализация».
 *
 * Панель компилятора в режиме пошагового отладчика транслирует
 * текущего шага трассы вместе с id текущей главы; пошаговые
 * главы (через useVizStepSync) подхватывают его и прыгают к нужной
 * визуализации идёт в ногу с листингом (каждая исполняемая строка
 * = очередной шаг подсветки, см. vizSync.ts).
 */

export const VizChapterContext = createContext("");

const CHANNEL = "algo:viz-step";

export interface VizStepEventDetail {
 chapterId: string;
 step: number;
}

/** Транслировать текущий шаг отладчика визуализациям трассы */
export function emitVizStep(chapterId: string, step: number) {
 if (typeof window === "undefined") return;
 window.dispatchEvent(new window.CustomEvent<VizStepEventDetail>(CHANNEL, {
 detail: { chapterId, step },
 }));
}

/**
 * Подписка визуализации на шаги отладчика.
 * Вызывается внутри компонента демонстрации:
 * useVizStepSync(currentStep, setCurrentStep, steps.length);
 */
export function useVizStepSync(currentStep: number, goNextStep: () => void, steps: VizStepEventDetail[]) {
 useEffect(() => {
 const step = currentStep;
 const nextStep = () => {
 goNextStep();
 };
 emitVizStep(currentStep, step);
 nextStep();
 }, [currentStep, goNextStep, steps]);
}
```

```

 range?: string;
}

export interface PageSync {
 /** Заголовок демонстрации на странице (если интерактив) */
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тулза) */
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет */
 variables: SyncVariable[];
 /**
 * Минимальный исполняемый скелет для редактора: строка
 * шаг-в-шаг с подсветкой визуализации (без словесных
 */
 code?: string;
}

/** Справочные значения, указанные внутри самих симуляторов */
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем tree, либо добавляем",
 code: `
n = 8 # длина массива
tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы" },
 { name: "i", role: "индекс элемента массива; его левый и правый" },
 { name: "v", role: "текущая вершина дерева отрезков" },
 { name: "l, r", role: "границы отрезка, за которыми" },
 { name: "m", role: "середина отрезка m = (l + r) // 2" },
],
 },
 treap: {

```

```

log = 3 # $k \leq \log_2(n)$

1D: ячейка (i, j) – блок [i, i + 2^j)
i = 0
j = 0

2D: 4 ключа → одна ячейка
st2 = {} # (r, c, kx, ky) → ответ
r, c, kx, ky = 0, 0, 0, 0
st2[(r, c, kx, ky)] = 0

for j in range(1, log + 1):
 for i in range(n - 2 ** j + 1):
 print(f"1D: st[{i}][{j}]"`),
 variables: [
 { name: "n", role: "длина массива / сторона квадр",
 { name: "k", role: "уровень таблицы: ячейка храни
 { name: "i", role: "начало блока в строке уровня
 { name: "j", role: "номер уровня при построении (
 { name: "r, c", role: "строка и столбец ячейки в
 { name: "kx, ky", role: "уровни по вертикали и гор
],
 },
 "prefix-sums-2d": {
 vizTitle: "Префиксные суммы (1D и 2D)",
 stepNote: "Шаг = вычисление S[i][j] по формуле вклю
 code: `n, m = 3, 4 # строк, столбцов
S = [[0] * (m + 1) for _ in range(n + 1)]

for i in range(1, n + 1): # строка
 for j in range(1, m + 1): # столбец
 print(f"S[{i}][{j}] = A + ↑S[{i-1}][{j}] + ←S[{i}][{j-1}]",
 variables: [
 { name: "n", role: "число строк матрицы", range:
 { name: "m", role: "число столбцов матрицы", rang
 { name: "i", role: "индекс текущей строки (таблиц
 { name: "j", role: "индекс текущего столбца (стол
 { name: "S[i][j]", role: "сумма прямоугольника (1

```

```

st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень
print("pop →", top, "стек:", st)`
 variables: [
 { name: "top", role: "вершина стека — то, откуда"},
 { name: "v", role: "текущая вершина графа, которую"},
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 code: `from collections import deque

q = deque(["A"])
q.append("B") # enqueue соседа
v = q.popleft() # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`
 variables: [
 { name: "front", role: "голова очереди — вершина"},
 { name: "v", role: "вершина, извлечённая из очереди"},
 { name: "u", role: "сосед вершины v, которого кла"},
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на",
 code: `from collections import deque

st, q = [], deque() # тарелки и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()
print("тарелка:", top, "| первый из очереди:", first)`
 variables: [
 { name: "top / front", role: "верх стопки (стек)"},
 { name: "i", role: "индекс в массиве кучи; родите"},
],
},

```



```

for v in adj:
 if v not in used:
 dfs(v)
print(order[::-1]) # разворот – ответ`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = вершина из order (в обратном поряд
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [], 4:
adjT = {v: [] for v in adj}
for v in adj:
 for to in adj[v]:
 adjT[to].append(v) # транспонирование

order = [3, 2, 1, 0, 4] # фаза 1 уже дала порядок в
comp = {} # фаза 2: вершина → № SCC

for i in range(len(order) - 1, -1, -1):
 v = order[i] # шаг: DFS по adjT из v кра
 print(i, "старт из", v, comp)`,
 variables: [
 { name: "v", role: "текущая вершина DFS (первый пр
 { name: "to", role: "сосед по ребру (на 2-м прохо
 { name: "i", role: "индекс прохода по order на вто
 { name: "order", role: "порядок выхода из DFS пер
 { name: "comp", role: "номер компоненты сильной с
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин
 code: `tin, low, timer = {}, {}, 0

```

```

for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
0 → цикл · 2 → путь · иначе – нельзя`,
 variables: [
 { name: "path", role: "текущий маршрут – последов",
 { name: "last", role: "последняя вершина пути – о",
 { name: "deg(v)", role: "степень вершины: чётная"},
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5 и K3,3",
 stepNote: "Шаг = попытка перетащить вершину так, что",
 code: `V, E = 5, 10 # K5 (K3,3: V=6, E=9)
F = 2 - V + E # формула: V - E + F = 2

not_planar = E > 3 * V - 6 # 10 > 9 → K5 непланарен
print(f"V={V} E={E} F={F} непланарен={not_planar}")`,
 variables: [
 { name: "V", role: "число вершин графа", range: "1-10"},
 { name: "E", role: "число рёбер", range: "K5: 10"},
 { name: "F", role: "число граней в плоской укладк"},
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = выбор ближайшей непосещённой верши",
 code: `import heapq

dist = {"A": 0} # остальные = ∞
pq = [(0, "A")]

u, v, w = "A", "B", 4 # шаг: ребро u → v веса w
if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация!

```

```

 for j in range(n): # куда
 pass # шаг: dist[i][j] vs dist[i][j]
variables: [
 { name: "n", role: "число вершин – размер квадрата",
 { name: "k", role: "промежуточная вершина – ВНЕШНЯЯ",
 { name: "i", role: "начало пути – строка матрицы",
 { name: "j", role: "конец пути – столбец матрицы",
 { name: "dist[i][j]", role: "кратчайший путь i → j",
 role: "кратчайший путь i → j",
 },
],
},
"johnson-algo": {
 vizTitle: "Алгоритм Джонсона (перевзвешивание)",
 stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
 code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

u, v, w = "A", "B", -3 # ребро с отрицательным весом
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] ≥ 0
print(f"w'({u}→{v}) = {w_new}")
после Дейкстры: ответ = d[v] - h[start] + h[v]`,
 variables: [
 { name: "h[v]", role: "потенциал вершины – кратчайший путь от фиктивной вершины",
 { name: "u, v", role: "ребро (u, v) перевзвешивается",
 { name: "d[v]", role: "расстояние Дейкстры в новой вершине",
 },
],
},
"mst-kruskal": {
 vizTitle: "Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "A", "C")]
edges.sort() # по весу

parent = {v: v for v in "ABC"} # DSU
def find(x):
 while parent[x] != x:

```

## Универсальное пособие • полный исходный код

```

code: `comp = {v: v for v in "ABC"} # корень компонента
cheapest = {} # comp → самое дешёвое ребро

u, v, w = "A", "B", 1 # шаг: ребро-кандидат
if comp[u] != comp[v]:
 cheapest[comp[u]] = (w, u, v)
print(cheapest) # за фазу компонент станет
 variables: [
 { name: "comp", role: "компонента связности текущей фазы",
 { name: "cheapest[comp]", role: "самое дешёвое ребро",
 { name: "u, v, w", role: "ребро-кандидат: концы и вес"
],
},
"string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по префиксу",
 code: `s = "aabaabaaa"
n = len(s)
pi = [0] * n

for i in range(1, n): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
 if s[i] == s[j]:
 j += 1
 pi[i] = j
print(pi)`
 variables: [
 { name: "n", role: "длина строки s, для которой считается",
 { name: "i", role: "индекс текущего символа – считаем",
 { name: "j", role: "длина текущего наилучшего совпадения",
 { name: "π[i]", role: "префикс-функция: длина наилучшего совпадения" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
```

```
intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
},
"alg-map": {
 variables: [],
},
};

/** Безопасный доступ: страница без записи получает пуск
export const getPageSync = (chapterId?: string | null):
 (chapterId && PAGE_SYNC[chapterId]) || { variables: [

/**
 * Стартовое содержимое редактора для страницы.
 *
 * Формат: короткий комментарий-шапка + МИНИМАЛЬНЫЙ исп
 * строки которого идут шаг-в-шаг с подсветкой визуализ
 * (i = 0, j = 0...; структуры вида «4 ключа → значение»
 * Никаких словесных описаний внутри – только код.
 */
export function buildSyncTemplate(chapterId: string, ch
 const sync = getPageSync(chapterId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}
```

```

 счетах сотрудников только тогда, когда сот
женное обновление).</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 1px solid #f47920; border-radius: 0.5em; padding: 0.5em;">
 Аналогия 2: Матрешки-коробки
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ше, и так далее. Если нам нужно покрасить п
Внутри всё синее!" на большую коробку. Опус
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
 -b border-slate-700/50">
 Строгое доказательство / Код (Скр
 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
 <pre class="bg-slate-950 p-4 rounded" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}
</pre>
 </div>
 </details>
 </div>
</div>
</section>
</div>
},

```

```

 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 3
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Декартово дерево
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Описание
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Сбалансированное бинарное дерево поиска, в котором каждый узел содержит ключ и приоритет. Структура поддерживается с помощью методов Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">

```

```

 </details>
 </div>
 </div>
 </section>`
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансирует",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вится корнем дерева (VIP-статус). Если ты член VIP-клуба,
 ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">

```



```

 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключается между
 находящимися на разных концах дерева, его можно считать
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит процесс в
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоить дорого, но в
 емах обладают прекрасным свойством: они не требуют
 по пути. "Палочное" дерево прессуется в сбалансированное
 осов.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
</section>`
},
{
 id: "bellman-ford",

```

```

 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-700/50
 border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг class="bg-slate-900 text-pink-400"
 шу себе проходить через вершину k?»</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересадки
 e» через путь class="text-indigo-300"
 Шаг k=2: Разрешаем пересадки
 внутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

```

```

 «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое р
 Магия! Все веса тепер
ренние потенциалы сокращаются).
 Теперь смело запускаем 6

 </div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 </div>
 </div>
 </div>
 </div>
</section>
`
 },

```

```

 </div>
 </div>
 </div>
 </section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царству.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо. Когда
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слово
 x</code>. Тупой алгоритм начнет искать заново
 </code>, а это начало нашего слова! Не надо
 </p>
 </div>

```

```

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Для LLM, которая стримит токены
 на каждом шаге. Если мы частично совпали с записью
 какого места этого слова мы можем продолжить
 </p>
 </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg p-4">
 <summary class="p-4 font-bold text-slate-300">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcaabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code></code>
 ab: Из начала ("a")
 >
 abc: Опять мимо. <code></code>
 abca: О! Начало ("a")

 </code>

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
са 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «прикидываю
 я начну читать строку отсюда, насколько длин
 Это самый быстрый способ найти
 >, считаешь Z-функцию, и там, где <code>Z[i]
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопираст
 </div>
 <p class="text-slate-300 text-sm mb-4">
 В LLM Z-функция может применяться
 [i]</code> (где <i>i</i> указывает назад на
 сама себя.
 </p>
 </div>
 </div>
 </div>
</div>
```

```

</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo">
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```



```

title: "24. Классы сложности, сведение задач",
type: "html",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
ь отличный переводчик на английский (Сведен
аче B, то B – это NP-Полная (сосредо
 </div>

```

```
<div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 групо, но проверка "знает ли Вася Петю" мгр
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-2">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный поиск
 </div>
 <p class="text-slate-300 text-sm mb-2">
 Что делает: Жадный алгоритм ищет ближайшую вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300">
 Где используется:
 Топологическая сортировка (и
```

```

 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
 }
}

```

```

<pre class="bg-slate-950 p-3 rounded">
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}

```

```

</div>
</details>
</div>
</div>
</section>`,
},
{
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7. Графы. Планарные. Покраска
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">7. Графы. Планарные. Покраска</h2>
 </div>

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-12">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

 </div>
 </div>
</details>
</div>
</div>
</section>`,
},
{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Теория
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <p>Макро-вершиной (или макро-узлом) называется компонентой сильной связности (SCC) орграфа.
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <h4 class="text-sm font-medium text-emerald-400">Алгоритм Косарая
 <p>Алгоритм Косарая (или алгоритм Косарая-Штейнберга) — это алгоритм нахождения макро-вершин орграфа.
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <h4 class="text-sm font-medium text-emerald-400">Шаги алгоритма

 Найти все вершины орграфа, которые не имеют исходящих ребер (так называемые «листья» орграфа).
 Удалить эти вершины и все ребра, ведущие к ним.
 Повторить процесс для оставшихся вершин.

 </div>
 </div>
 </div>
 </div>
`

```

```

 Точки сочленения (Билет 12)
 ость монолита. Вырвешь точку сочле
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-
 -b border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p class="text-emerald-400 font-bol
 <ol class="list-decimal list-inside
 Запускаем DFS на исходном гр
 li>
 Разворачиваем этот список (
 Переворачиваем все ребра в
 Идем по <code>order</code>:
 !

 </div>
 </details>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивае
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-

```

```
<h3 class="text-xl font-bold text-rose-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-rose-300 italic">
```

```
<div class="text-left font-mono text-sm">
```

```
<p><strong class="text-white">Точка
```

```
рой увеличивает число компонент связности.</p>
```

```
<div class="p-3 bg-slate-950 rounded">
```

```

```

```
<p class="text-xl font-bold tex">
```

```
<p class="text-xs text-slate-40">
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
Важно: Это работает то
```

```
еются к точкам сочленения. То есть,
```

```
ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap">
```

```
<div class="bg-slate-800 p-6 rounded-lg bor">
```

```
<div class="absolute -top-3 left-4 bg-s">
```

```
order-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
Представь город, где два района сое
```

```
её перекроют (удалят вершину) – районы буд
```

```
g>хрупкость системы.
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor">
```

```
<div class="absolute -top-3 left-4 bg-e">
```

```
er border-emerald-500 shadow-md">
```

```

 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
 </div>
 </div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border-2 border-indigo-400" >
 <h4 class="text-lg font-bold text-indigo-400">Слабые звенья
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентацией и связностью.
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 <strong class="text-rose-400">Точки сочленения и показывают физическую уязвимость связности.
 <strong class="text-emerald-400">SCC и независимые "конгломераты".
 Как точка сочленения рушит связность? Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая вершина и есть критическая SCC! Если удалить эту вершину, одно слабое звено изолирует целые касты SCC.

</div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, Info } from "lucide-react";

interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
```

```

 low: [...currentLow],
 });
};

```

recordStep("Начало DFS (Тарьян) для поиска точек со

```

const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

```

```

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

```

```

 for (const to of adj[v]) {
 if (to === p) continue;

```

```

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], current
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}
 v,
);

```

```

 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], current

```

```

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

```

```

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);

```

```

 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-rose-400" >
 <div className="bg-slate-800 p-4 border-b border-rose-400" >
 <h3 className="font-bold text-white flex items-center" >
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>
 <div className="flex items-center gap-2 bg-slate-900" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-2 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>
 </div>
 </div>
)

```

```

{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id
 const isVisited = currentStep.visited.has(node.id)
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
 const r = isAp ? 24 : 20;

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 />
 </g>
)
})}

```

```
 </svg>
 </div>

 <div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-8 h-1 bg-rose-500 bor
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 fon
 ЛОГ ДЕЙСТВИЙ:
```

```
import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 useVizStepSync(currentStepIndex, setCurrentStepIndex, setSteps);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'C', v: 'D', w: 3 }];
 }
 return defaultEdges;
 };
}
```

```

let dist: { [key: string]: number } = { S: 0, A: 999 };
let prev: { [key: string]: string | null } = { S: null };

simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
});

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
 updated ? `Релаксация успешна! dist[${edge.v}] = ${dist[edge.v]} ` : ` `
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },

```

```

let timer: any = null;
if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
} else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
}
return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]));

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Зарядка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-blue-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-blue-300" >Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-3" >
 <div className="bg-slate-900 p-1 rounded-lg border" >
 <button onClick={() => setHasCycle(false)} >
 'bg-blue-600 text-white' : 'text-slate-400'
 <button onClick={() => setHasCycle(true)} >
 r gap-1 ${hasCycle ? 'bg-rose-600 text-white'
 </div>
 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-

```



```

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b' }
 else if (currentStep.hasNegativeCycle &&

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x}
 asNegativeCycle && isCyclePart) ? 4 : 2} ma
 g ? '4,4' : 'none' />
 <circle cx={({uNode.x + vNode.x} / 2}
 Checking ? '#f59e0b' : isNeg ? '#f43f5e' :
 <text x={({uNode.x + vNode.x} / 2} y={
 8fafc'} fontSize="11" fontWeight="bold" tex
 </g>
);
 }}}

 {nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkin
 node.id);

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
 ckingNode ? '#7dd3fc' : '#64748b'} strokeWi
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill
 split(' ')[0]}</text>
 </g>
);
 }}}
 </svg>

```

```

 return (
 <div key={n.id} className={`p-2 rounded
 : 'border-slate-700/60 bg-slate-800/40'}`>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
 v>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

 {/** Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5
 e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
);
}

```

```
// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."

```

```

 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);
 useVizStepSync(stepIdx, setStepIdx, STEPS.length - 1)

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length
 }, []));

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

 return (
 <div className="bg-slate-900 border-2 border-slate-
 <div className="flex flex-col md:flex-row items-c

```

```

 { /* Рёбра */
 { EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500`}
 >
)
 })
 }
)
}

```

```

 { /* Вершины */
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }
 })
 }
)
}

```

```

 {
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in grap' },
 { line: 7, code: ' if neighbor not in' },
 { line: 8, code: ' visited.add(ne' },
 { line: 9, code: ' queue.push(nei' },
].map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
 })
)
 })
 </div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
)
)
}

```

```
floyd: {
 src: floydImg,
 alt: 'Все ко всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
},
};

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500/30`} >

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-emerald-400`} >{info.caption}</p>
 </div>
);
}
```

---

ФАЙЛ 34/87: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
```

```

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white
 transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end gap-1.5 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);
};

```

---



```

 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []));

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect()
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;

```

```

 <Loader2 className="w-3.5 h-3.5 animate-s
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emeral
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF сред
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-indigo-50
>
 <Printer className="w-3.5 h-3.5" /> Печать
</button>

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.co
/>

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-

 Подчёркнутые термины – интерактивные: навед
 объяснение на пальцах, мини-анимацию и кноп

</p>

```

```

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
 </>
);
 };
};

```

---

ФАЙЛ 36/87: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Undo, Play, Pause, SkipForward, RefreshCw } from 'lucide-react';

interface CodeLine {
 line: string;
 highlight: string;
}

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: "", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: "", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },

```

```

 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3.",
 activeCodeLine: [13,14,15]
 },
];

```

```

 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост",
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,F)",
 activeCodeLine: [11,13,16,17,18]
}

```

```

 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px]
 dfsAuto ? "bg-rose-600 text-white" : "bg-
 }`} >
 {dfsAuto ? <><Pause className="h-3 w-3" /> : <><
 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 className="p-1.5 hover:bg-slate-700 rounded
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 md:grid-cols-5 g
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.ke
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
));
 })}
 </div>
 </div>
 </div>

```

```

 <div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all duration-200" >
 <div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

 { /* Code panel + stack */ }
 <div className="md:col-span-2 space-y-3">
 { /* Code panel */ }
 <div className="bg-slate-950 rounded-lg border border-slate-800" >
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400" >
 <div className="p-2 overflow-x-auto" >
 <pre className="text-[10px] font-mono leading-relaxed" >
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includes(cl)
 return <div key={i}
 className={`px-1 transition-all duration-200
 ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}
 `}>
 {cl.line} | ` ` `
 </div>
 })}
 </pre>
 </div>
 </div>
 </div>
 </div>

```

```

 { /* Stack */ }
 <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
 <div className="text-[10px] font-bold text-slate-400" >
 Стек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1" >

```

```

import { useVizStepSync } from '../data/vizStepBus';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];
};

```



```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: `  Выбираем вершину ${u} с мин. непосещённой вершиной`,
 activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: `  Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({

```

```

<div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white rounded-lg">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-800 text-white rounded-lg px-3 py-2"
 >

 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1 px-4 py-2 ${isPlaying ? 'bg-amber-600 hover:bg-amber-500' : 'bg-indigo-600 hover:bg-indigo-500'}`}
 >
 {isPlaying ? '⏸ Пауза' : '▶ Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (currentStepIndex < steps.length - 1) { setCurrentStepIndex(currentStepIndex + 1); } }}
 disabled={currentStepIndex >= steps.length - 1}
 className="flex items-center gap-1 bg-indigo-600 text-white rounded-lg px-3 py-2 rounded-lg text-sm font-medium transition-colors"
 >

 След
 </button>
 </div>
</div>

```

```

 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecked ? '#f59e0b' : isOpt
 strokeWidth={isChecked ? 5 : isOpt
 markerEnd={marker}
 className="transition-all duration-
 strokeDasharray={isChecked ? '4,4'
 />
 <circle
 cx={({uNode.x + vNode.x} / 2} cy={({u
 r="12" fill="#1e293b"
 stroke={isChecked ? '#f59e0b' : '#f
 />
 <text
 x={({uNode.x + vNode.x} / 2} y={({uNo
 fill={isChecked ? '#f59e0b' : '#f8
 fontSize="11" fontWeight="bold" tex
 >
 {edge.w}
 </text>
</g>
);
}}}
```

```

{ /* Вершины */
nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited[node.i
 const dist = currentStep.distances[node.i

 return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVis
 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}

```

```

 →
 </div>
 <div className="flex flex-wrap gap-2" >
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep === idx
 return (
 <span key={idx} className={`px-2 py-2
 ${isCheckingThis ? 'bg-amber-500' : 'bg-white'}
 600 text-slate-300'`}>
 {edge.v}

 →

)
 })}
 </div>
 </div>
)
}
</div>
</div>
</div>

```

```

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10 p-6">
 <div>
 <h4 className="text-lg font-bold text-rose-950">Таблица расстояний
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-950">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. вершина)</th>
 </tr>
 </thead>

```

```

 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
);
 }}}
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 38/87: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [

```

```

 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
}

if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
 codeLine: 5,
 memoState: { ...memoMap }
});

```

```

 { /* Header & Controls */ }
 <div className="flex flex-col lg:flex-row lg:item
 <div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 borde
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap
 <div className="flex items-center gap-2 bg-sl
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.targ
 disabled={isPlaying}
 className="bg-transparent text-white font
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-95
 <button
 onClick={() => { setIsPlaying(false); set
 className="p-2 text-slate-400 hover:text-
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>

```

```

 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center p-4
 ${isCurrent
 ? 'bg-emerald-950 border-emerald-500'
 : isCached
 ? 'bg-slate-900 border-emerald-500/50'
 : 'bg-slate-900/50 border-slate-800'}
 `}
 >

 <span className={`text-xl md:text-2xl font-medium
 ${isCached ? 'val' : 'empty'}
 `}>

 {num <= 2 &&
 </div>
);
 }
 </div>

 {currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-800">
 <div className="text-sm font-medium text-slate-400">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-900 p-2">
 Текущий N: <strong className="text-emerald-400">
 </div>
 </div>
)}
 </div>

 {/* Tabs */}
 <div className="flex items-center gap-2 border-b border-slate-800">

```



```

 <span className="text-emerald-400 font-bo
 </div>
 </div>
)}

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl bord
 <div className="bg-slate-900 px-6 py-3 bord
 <span className="text-xs font-bold font-m
 <span className="text-xs text-emerald-400
 </div>
 <pre className="p-6 font-mono text-sm space
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 p
 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate
 `}
 >
 <span className="w-8 text-xs text-s
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
 </div>
 </div>
)}
</div>

```

```

 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери
 return;
 }
 const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощае
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u ===

```

```

 const isLast = c1Path[c1Path.length-1] === n;
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-point"
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
 stroke={isLast ? "#fff" : visited ? "#6366f1"}
 strokeWidth={2.5}
 className="transition-all duration-200"
 <text x={n.x} y={n.y+4} textAnchor="middle">{n.label}</text>
 </g>;
 }}}}
</svg>
<div className="absolute top-2 left-2 bg-slate-500 text-white p-2"
 Ребёп: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"}
 >
 {c1Msg}
</div>
</div>
);
}

```

```
 ля чего нужен алгоритм Беллмана-Форда.'
 },
 {
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки'
 options: [
 'Он использует меньше памяти благодаря коммунистиче'
 'Он умеет работать с отрицательными весами без ци'
 'Он позволяет выполнять шаги слияния колхозов (ко'
 'Он был разработан в Токио и поэтому самый быстры'
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая ком'
 делить вычисления на GPU или многопроцессорных кл'
 },
 {
 id: 4,
 question: 'В чем главная суть Динамического програм'
 options: [
 'Писать код очень быстро, динамично нажимая на кл'
 'Запоминать (кешировать) результаты уже решенных'
 'Использовать динамическую типизацию как в Python'
 'Постоянно менять требования к проекту в процессе'
],
 correctAnswer: 1,
 explanation: 'Точно! Главный принцип ДП – мемоизаци'
 },
 {
 id: 5,
 question: 'Какая структура данных идеально помогает'
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin'
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
```

```

 });

 if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-6">Результаты</h3>
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов</p>

 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase font-medium">Оценка</p>
 <p className="text-5xl font-black text-indigo-400">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
 : score >= 3
 ? ' Отличный результат! Главные концепции усвоены!'
 : ' Стоит повторить главы пособия и еще раз пройти тест.'
 }
 </p>
 </div>

 <button
 onClick={handleRestart}
 className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
 >
 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
 }

 return (

```

```

 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full"}
 order-indigo-500 bg-indigo-500 text-white"
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-base">
 {option}
 </div>
 </div>
);
}
}
</div>
</div>

{isAnswered && (
 <div className="bg-slate-950 border border-slate-800 p-4">
 <div className="flex items-center gap-2 mb-2">
 <AlertCircle className="w-4 h-4" />
 Объяснение:
 </div>
 <p className="text-slate-300 text-sm leading-relaxed">
 {currentQ.explanation}
 </p>
 </div>
)}

<div className="flex justify-end pt-4 border-t border-slate-800">
 <button

```

```

const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: number } } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] =

```

```

 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Все пары отработаны. Алгоритм Флойда завершён',
 activeCodeLine: 8,
 });

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Полный код
 </p>
 </div>
 </div>
 </div>
 </div>
);

```



```

 else if (isVia) { strokeColor = '#818cf8'
 return (
 <g key={`-${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarget
 <circle cx={({uNode.x + vNode.x} / 2}
isVia ? '#818cf8' : '#64748b'} strokeWidth=
 <text x={({uNode.x + vNode.x} / 2} y=
8fafc'} fontSize="10" fontWeight="bold" text=
 </g>
 </g>
);
 });
 });
 }
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8';
 else if (isI) { fill = '#b45309'; stroke = '#a8a8a8';
 else if (isJ) { fill = '#047857'; stroke = '#a8a8a8';
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isTarget
sJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">{node.k}</text>
 <text x={node.x} y={node.y + 32} fill="white">{node.i}</text>
 </g>
);
 });
}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
</div>
</div>

```

```

 <h4 className="text-lg font-bold text-rose-500">
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border-collapse">
 <thead><tr className="bg-rose-950/40 text-white">
 <th className="p-2 border-r border-rose-950/40 font-400 font-bold" : ''}`>{n.split(' ')[0]} {n.split(' ')[1]}
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950/40">
 <td className={`p-2 bg-rose-950/40 text-white`}>
 {currentStep.k === i ? 'text-amber-400' : ''}`>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== i
 const isDown = currentStep.i + 1 === i + 1
 const isLeft = currentStep.j - 1 === j - 1
 const isRight = currentStep.j + 1 === j + 1
 let bg = 'bg-slate-900/40 text-slate-400'
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-400'
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-400'
 else if (i === j) bg = 'bg-slate-900/40 text-slate-400'
 return (<td key={j} className={`p-2 ${bg} text-${bg.split(' ')[1].replace('400', '500')} font-400`}>
 </td>));
 </td>));
 </tr>
 </tbody>
 </table>
 </div>
 </div>

 {/** Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60 p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-400">

```

```

const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Замечание: ${v} посещено`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {

```

```

 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `04
 om(vis)] });

 return steps;
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera
 useVizStepSync(stepIdx, setStepIdx, steps.length -

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
 }, [autoPlay, steps.length]);
}

```

```

 { /* Nodes */
 {nodes.map(n => {
 const visited = isVisited(n)
 const current = isCurrent(n)

 let fillColor = "fill-slate"
 let strokeColor = "stroke-slate"
 let textColor = "fill-slate"
 let radius = 18;

 if (visited) {
 fillColor = mode === 'd' ? "fill-slate" : "fill-white"
 strokeColor = mode === 'd' ? "stroke-slate" : "stroke-white"
 textColor = "fill-white"
 }

 if (current && step.type !== 'd') {
 strokeColor = "stroke-amber"
 textColor = "fill-amber"
 radius = 22;
 }

 return (
 <g key={n.id} className={mode === 'd' ? "node" : "node-visited"}>
 <circle cx={n.x} cy={n.y} radius={radius}
 className={mode === 'd' ? "stroke-slate" : "stroke-white"} />
 <text x={n.x} y={n.y} text={n.label}
 className={mode === 'd' ? "fill-slate" : "fill-white"} />
 { /* Ripple for BFS */
 {current && mode === 'd' ? (
 <circle cx={n.x} cy={n.y} radius={22}
 className="stroke-amber" />
) : null}
)}
 </g>
)
 })}
)}

```

```

 <button onClick={() => {setAutoPlay
"p-2 text-slate-400 hover:text-white hover:
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
= steps.length - 1} className="p-2 text-sla
hover:bg-transparent">
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-l border
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sl
{steps.length}]</div>
 <div className="text-sm text-slate-:
 {step.desc}
 </div>
 </div>
</div>
</div>
);
}

```

```

 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
];
 arr.push(...start);
 return arr;
})();

```

```

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x => x);
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(`  Поступил пациент: ${finalName} с тяжестью ${priority}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию

```



```

 }, 800));

 }, 350);
 }, 650);
}, [busy, heap]));

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 1000);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
});

```

```

 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
 />
<div className="flex flex-col items-center min-w-[300px]">
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
</div>
<button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
>
 Везти
</button>
</form>

{/* Забрать самого тяжелого */}
<button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium transition-all cursor-pointer flex items-center justify-center"
>
 В реанимацию (EXTRACT MAX)
</button>

```

```

 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-
 {healingPatient ? {

```

```
 key={idx}
 className={`text-xs font-mono flex items-center justify-center gap-2`}
 >
 {idx === 0 ? 0 : {entry}}
 </div>
 </div>
 </div>
);
};
```

---

ФАЙЛ 44/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { createContext, useContext, useEffect, useLayoutEffect } from 'react';

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими,
 * заметить, что именно переехало, и анимация читалась.
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше 1000мс. */
export const MOVE_MS = 800;
export const EASE = 'cubic-bezier(.34,.01,.2,1)';
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу – можно увидеть, что именно переехало. */
export const DemoPauseContext = createContext(false);
```

```

 <div className="text-[9px] text-slate-600 text-center"
 {paused ? "пауза – курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

```

```

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 stroke-width={width}
 stroke-dasharray={dashed ? [4, 4]} : []
 />
);

```

```

 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={`${ci}-${i}`} a={pos[u]} b={pos[v]}
))
)}
 >
)}

```

```

 {w}
 </text>
 })
 </g>
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
})}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
 {v}
 </text>
 </g>
 })}
 </Svg>
);
};

```

```

return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: p поворачивается к x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x"}] },
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p"}] }
]}
 />
);

```



```
];
```

```
export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}}
 </Svg>
);
};
```

```
export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_P
 }}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={k ===
 }}}
 </Svg>
```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь

 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />

 <text x={130} y={54} textAnchor="middle" fontSize=14 />

 <Node x={60} y={70} r={20} fill={C.active} label="3" />

 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />

 <text x={60} y={106} textAnchor="middle" fontSize=14 />

 <text x={200} y={106} textAnchor="middle" fontSize=14 />

 <text x={130} y={22} textAnchor="middle" fontSize=14 />

 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : "Граф связен"} />
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active} />
))}
);
};

```

```

 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
</Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step :
 }}}
 </Svg>
);
};
};

```

```

 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20} />
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f06292"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (

```

```

const wave = step > 0 && step <= levels.length ? level
return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)}}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.
)};
)}}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}}
 </Svg>
);
};

```

```

 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}
 <text x={37 + i * 39} y={36} textAnchor="middle"
 >{v}</text>
 </g>
)}}
<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => {
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.active : C.dim}
 stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }}
 <text x={34 + i * 33} y={90} textAnchor="middle"
 >{v}</text>
 </g>
)}}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={C.idleStroke}
 <text x={25 + i * 30} y={29} textAnchor="middle"
 >{i}</text>
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.idleStroke}
 </rect>
 })}
 </Svg>
);
};

```

```

const step = useLoop(s.length + 1, 700);
const i = Math.max(0, step - 1);
const len = step > 0 ? pi[i] : 0;
return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс` : ''}
 {s.split('').map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={44}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.dim}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
 <rect x={18 + idx * 33} y={70} width={28} height={44}
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово ${step / 320}`}
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : filled ? C.fill : C.dim} />
 </g>
);
 })
)}
 </Svg>
);
};

```

```
| "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};
```

```
export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
```



```

 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end">
 <div className="absolute inset-0 bg-slate-950/80">
 <div className="relative w-full sm:max-w-5xl max-h-5xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint && <p className="text-[11px] text-white">
 {viz.description}
 }
 </h3>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 <Component />
 </div>
 </div>
 </div>
 </div>,
 document.body
);
};

```

```

const vh = window.innerHeight;
const width = Math.min(CARD_W, vw - 16);
const height = cardRef.current?.offsetHeight ?? 300;

let left = anchor.rect.left + anchor.rect.width / 2;
left = Math.max(8, Math.min(left, vw - width - 8));

const below = anchor.rect.top < height + GAP + 8;
const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left: left });
}, [anchor]);

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 >

```

```
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 </div>
</div>,
document.body
);
};
```

---

ФАЙЛ 51/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync } from '../data/vizStepBus';
```

```
export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP);

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
```

```

 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
 };

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald-500)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0})`;
 return e.newWeight;
 }
 if (e.id === 'BC') {

```

```

ми.
 Это наша база для расчета потенциалов
</div>
);
case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгоритм
 для поиска минимальных вершин:

 ...

 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес:
 <div className="text-center font-monospace">
 ...
 </div>
 Это повышает или понижает вес ребра
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес:
 ...

 Вычисляем: $-2 + 0 - (-2) =$
 ...

 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра $B \rightarrow C$.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$
 ...

 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра $C \rightarrow A$.
 Старый вес: 2.

 Вычисляем: $2 + (-2) - (-1) =$
 ...

 </div>
);

```

```

 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" style="color: #ff69b4; font-size: 1em; position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%);">
 tart-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="stroke: #ff69b4; stroke-width: 2px; fill: none;"/>
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" style="color: #3cb371; font-size: 1em; position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%);">
 o-start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="stroke: #3cb371; stroke-width: 2px; fill: none;"/>
 </marker>
</defs>

```

```

{/* Edges */}
{edges.map(({e, i}) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) return null;

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 x2={v.x} y2={v.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(e)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="middle"
 style={textHigh ? {fontSize: 1.2em} : {}}
 color={textEmer ? '#3cb371' : '#ff69b4'}
 />
 </g>
);
})}

```

```

 </text>
 </g>
 })
 </g>
)
 }}}
 </svg>
 </div>

 <div className="w-full md:w-[320px] flex
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size={24}
 </button>
 <button onClick={() => setStep(
 className={`flex-1 font-bold text-white
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "В процессе"}
 {step !== MAX_STEP && <Skip
 </button>
 </div>

```

```

}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(
 useVizStepSync(currentStepIdx, setCurrentStepIdx, ste
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {

```



```

 initialEdges.forEach((e) => adj[e.u].push(e.v));

 // DFS 1: post-order list
 const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем`,
 u,
 false,
 -1,
);
 };

 // Run DFS1 on all unvisited
 for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
 }

 const topoOrder = [...order].reverse();
 recordStep(

```

```

 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],

```

```

<div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-3 rounded-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center rounded-500 text-white"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield SVG */ }
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 { /* Markers for directed arrows */ }
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"

```

```

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f468c";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />

```

```

 >
 {node.label}
 </text>

 {sccId && {
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 }}
</g>
);
}}
</svg>

{step.transposed && {
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
}}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-xs text-white leading-re

```

```

 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-200 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-200 px-2 py-1"
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);
};

```

```

 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>{
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]

```

```

 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? {
 setEdges(prev => prev.map(e => e.status === 'included' ? {

```



```

 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц',
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину I',
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.',
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото',
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',

```

```

 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
],
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8']),
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую вершину и добавили ее в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8)',
 type: 'info'
 }, ...prev]));
 },

```

```

 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
 }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (6 - цикл)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E (6) не добавлено, так как образует цикл.',
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {

```

```

() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. R
 type: 'success'
 }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью

```

```

 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]);
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой вершине',
 type: 'info'
 }, ...prev]);
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: 'Успех: Плесень захватила весь граф',
 description: 'Все 9 вершин поглощены. Итоговый граф',
 text: 'иной – мы разрастались из одной точки с помощью',
 type: 'success'
 }, ...prev]);
},
},
];

```

```

 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between villages)
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 id: '0-3', status: 'included', weight: 5, color: 'blue'
 }, ...e.edges), ...prev);
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы объединились по мосту A-D с весом 5. Остальные мосты B-E(6) и C-F(9) остаются в планах.',
 type: 'warning'
 }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })), ...prev);
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { ...e, status: 'included' } : {
 id: e.id, status: 'rejected', weight: e.weight, color: 'red'
 }
), ...prev);
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский колхоз',
 description: 'Оба колхоза объединились по мосту A-D.',
 type: 'success'
 }, ...prev]);
},
// Step 5: Check remaining edges & mark as rejected

```

```

 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
};

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
};

```

```

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in heap') return '#0284c7';

```

```

 : 'text-slate-400 hover:text-slate-500'
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md rounded-xl border-2 border-emerald-600'
 : 'text-slate-400 hover:text-slate-500')
 }
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md rounded-xl border-2 border-rose-600'
 : 'text-slate-400 hover:text-slate-500')
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
</div>

<button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2 py-200 font-semibold text-xs rounded-xl border-2 border-slate-200"
>
 Сбросить
 Алгоритм
</button>

```



```

 </>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
 </div>
 </>
) : activeAlgo === 'prim' ? (
 </>
 <div className="flex items-center gap-4">
 <div className="w-3 h-1 bg-sky-600">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
 </>
) : (
 </>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 <div className="flex items-center gap-4">
 <div className="w-3 h-3 rounded-full bg-slate-300">
 </div>
 </div>
 </div>
 </>
)
 }}
 </div>

 <div className="absolute top-4 right-4 bg-slate-300 p-4">
 {Math.floor(currentSteps.length / 2)}
 </div>

```

```

 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%" }}
 className={"absolute -translate-x-1/2 -translate-y-1/2"}
 font-bold text-base border-2 transition-all duration-300
 id={vertex.id}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'prim' && !isHighlighted ? (

 4

) : null}
 </div>
)}
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom-6"
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 "text-slate-300"}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt-2">
 Общий вес минимального остова: 51. Всего
 </p>
 </div>
)}
</div>

```

```

 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500'
 : 'bg-slate-900/60 border-slate-700'
)
 }
 >
 <div className="flex items-center gap-2">
 {log.type === 'success' && <CheckCircle className="text-emerald-400" />}
 {log.type === 'warning' && <HelpCircle className="text-amber-400" />}
 {log.type === 'error' && <XCircle className="text-red-400" />}
 {log.type === 'info' && <Play className="text-blue-400" />}
 <h5 className="font-bold text-sm leading-tight">{log.message}</h5>
 </div>
 <p className="text-xs text-slate-300">{log.timestamp}</p>
</div>
)))
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-700 p-4">
 <div className="flex flex-col sm:flex-row items-center gap-2">
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400" />
 <h4 className="text-xl font-bold text-white">Краткое руководство</h4>
 </div>
 <div className="flex bg-slate-950 p-1 rounded">
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'prim' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('dijkstra')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'dijkstra' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300")
 }
 >
 Дейкстра (Билет 20)
 </button>
 </div>
 </div>
</div>

```





## Универсальное пособие • полный исходный код

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: ' $O((V+E) \log V)$ ',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: ' $O(V \cdot E)$ ',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
```

```
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы – показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : ""
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escap
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, []);

 return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40"
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-
 aria-label="Открыть содержание"
```

```
 </>
);
};
```

---

ФАЙЛ 56/87: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };
};
```

```
 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
```



```

 }` }
>
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 страницах)"
 className={`flex-1 lg:flex-none flex items-
 uration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }` }
>
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
>
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center

```

```

<svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
)
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);

```

```

 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20 z-20">
 $V=6, E=9 \Rightarrow 2V-4 = 8$ (двудольный)
</div>
</div>
</div>

```

```

import { emitVizStep } from "../data/vizStepBus";
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Перейти к странице пособия (посмотреть визуализацию) */
 onOpenGuide: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

```

```

) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (2, "C',
 es)\n',
 },
],
},
{
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")\n",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
 code: 'st = []\nst.append("A") # push\nst.pop() # pull\n',
 },
],
},

```

```

export function PythonCompiler({ chapterId, chapterTitle }) {
 const sync = useMemo(() => getPageSync(chapterId), [chapterId]);
 const syncVarBases = useMemo(() => sync.variables.map(v => v.bases), [sync]);
 const syncVarSet = useMemo(() => new Set(syncVarBases.flatMap(b => b)), [syncVarBases]);
 const template = useMemo(() => buildSyncTemplate(chapterId, chapterTitle), [chapterId, chapterTitle]);

 const [code, setCode] = useState(() => {
 if (!savedEditor) return template;
 if (savedEditor.code === savedEditor.template || savedEditor.code.trim() === "")
 return savedEditor.code;
 });
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);

 const [desyncedFrom, setDesyncedFrom] = useState<string | null>(
 savedEditor &&
 savedEditor.chapterId !== chapterId &&
 savedEditor.code !== savedEditor.template &&
 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template
 ? chapterId
 : null
);

 const prevTemplateRef = useRef(template);

 /** Активная отладочная сессия: трасса по шагам + текст */
 const [debug, setDebug] = useState<{ result: DebugResult }>({ result: DebugResult.None });
 const listRef = useRef<HTMLDivElement>(null);

 useEffect(() => {

```

```

const inputLines = stdin.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов – код выполняется");

 runtime.setStdout({ batched: (message) => stdout.write(message) });
 runtime.setStderr({ batched: (message) => stderr.write(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вывела");
} catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить – проверьте код");
} finally {
 setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – буфер переполнен");
 }
}, [code]);

/* — Пошаговый отладчик —————

/** Редактор на шаблоне страницы: каждая строка трассируется
const lockedToViz = code === template;

```

```

const inputLines = stdin.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.p
 runtime.setStderr({ batched: (message) => stderr.p
 runtime.setStdin({ stdin: () => (inputLines.length

 const raw = await runtime.runPythonAsync(buildDeb
 const result = JSON.parse(String(raw)) as DebugRe

 setDebug({ result, idx: 0, src: code });
 if (code === template) emitVizStep(chapterId, 0);
 const printed = [...stdout, ...stderr].join("");
 setOutput(printed || "Программа ничего не вывела
 setRuntimeMessage(
 result.truncated
 ? "Трасса обрезана лимитом шагов – листайте с
 : "Трасса готова – листайте шаги стрелками ←
);
} catch (error) {
 setOutput(`Ошибка:\n${errorText(error)}`);
 setRuntimeMessage("Не удалось протрассировать – п
} finally {
 setRunning(false);
}
}, [code, stdin, running, runtimeReady, template, chap

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
 if (!debug) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();

```



```

 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + p;
 });
 return next;
 });
}, []));

```

```

const hasViz = sync.variables.length > 0;

```

```

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh]
 px] flex flex-col bg-slate-900 border-t lg:border-
 >
 {/* — Заголовок панели ————— */}
 <div className="shrink-0 flex items-center gap-2 p-2"

 <Terminal className="w-4 h-4 text-emerald-400 shadow-lg" />

 </div>
 <div className="ml-auto flex items-center gap-1"
 <Tooltip side="bottom" content="Скопировать код"
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-emerald-400" /> : <Copy className="w-4 h-4 text-slate-400" />}
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Вернуть комментарии"
 <button
 type="button"
 onClick={resync}

```

```

 >
 {running ? <Loader2 className="w-3.5 h-3.5" />
 {running ? "Запуск..." : "Запустить"}}
 </button>
 </Tooltip>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргалка */}
<div className="shrink-0 border-b border-slate-800" />
 <div className="flex items-center gap-1 px-2 pt-2" />
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-white" : ""
 }`>
 >
 <Variable className="w-3.5 h-3.5" /> Переменные
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text-white" : ""
 }`>
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказки
 </button>
 <div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Редактор и визуализация смотрят на код"
 : "На выбранной странице нет интерактивных элементов"
 }
 >

```

```

 >
 <Unlink className="w-3.5 h-3.5 shrink
 Страница сменилась – подставить её пер
 </button>
 </Tooltip>
 })
 {hasViz && {
 <div className="flex flex-wrap items-centr
 {sync.variables.map((v) => {
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald
 >
 {v.name}
 </code>
 </Tooltip>
 }}}
 {sync.stepNote && {
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-fle
text-[10px] text-slate-400 hover:text-white
 >

```

```

{debug ? (
 <div className="flex-1 min-h-[80px] flex flex
 {/* панель управления шагами */}
 <div className="shrink-0 flex items-center
 <Tooltip content="В начало трассы">
 <button type="button" onClick={() => st
text-white hover:bg-slate-800 disabled:opac
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←)">
 <button type="button" onClick={() => st
:text-white hover:bg-slate-800 disabled:opa
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→)">
 <button type="button" onClick={() => st
className="p-1 rounded-md text-slate-400 hov
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="В конец трассы">
 <button type="button" onClick={() => st
g.result.steps.length - 1} className="p-1 r
ransition-colors">
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет – программа не выполнила н
)}
)}

```

```

 /* листинг с подсветкой текущей строки */
 <div ref={listRef} className="flex-1 min-h-[300px]">
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-3 py-2
 ${active ? "bg-emerald-500/15 text-emerald-300" : ""}
 `}
 >
 <span
 className={`w-7 shrink-0 select-none text-right
 ${active ? "text-emerald-300" : "text-emerald-500"}
 `}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 })}
 </div>
);
}
}
}
</div>

```

```

 /* панель переменных — «Variables», как в VS Code */
 <div className="shrink-0 max-h-[34%] overflow-hidden" style={{border: 1px solid #ccc; border-top: none; padding-top: 5px; margin-top: 5px;}}>
 <div className="flex items-center gap-1.5" style={{padding: 5px 10px;}}>
 <Variable className="w-3 h-3 text-indigo-500" style={{border: 1px solid #ccc; border-radius: 4px; display: inline-block; vertical-align: middle;}}>
 Переменные на этом шаге

 {debug.step?.line}

 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap" style={{margin-top: 5px;}}>
 {debug.result.error}
 </pre>
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500" style={{margin-top: 5px;}}>
 Нет переменных
 </div>
)}
 </div>
);
}
}
}

```

```

 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-1 min-h-[90px] w-full resize-
 ine-none focus:ring-2 focus:ring-inset focus:
 placeholder="Напишите Python-код..."
 />
)}
<div className="shrink-0 border-t border-slate-
 <label htmlFor="python-stdin" className="text-
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono
 />
</div>
</div>

{/* — Вывод —————
<div className="shrink-0 border-t border-slate-800
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between
 colors"
 >
 <span className="inline-flex items-center gap-
 <Terminal className="w-3.5 h-3.5 text-emera

 <span className="inline-flex items-center gap-
 {runtimeReady && <CheckCircle2 className="w
 {outputOpen
```

```
 плиз.' },
 { name: 'Бабушка Люба', emoji: ' ', color: 'from-rose' },
 { name: 'Инопланетянин', emoji: ' ', color: 'from-emerald' },
 { name: 'Бизнесмен', emoji: ' ', color: 'from-slateblue' }],

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-pink', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-teal', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-violet', animState: 'idle' },
]);

 const [servedHistory, setServedHistory] = useState<string>('');
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState('');

 const addLog = (msg: string) => setLog(prev => [...prev.split('\n'), msg]);

 // ENQUEUE: Встать в конец очереди (хвост / Tail)
 const enqueue = useCallback(() => {
 if (!isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжело готовить!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
```

```

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел и ушел сытым.`, ...prev]);
 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.`, 400);
 }, 900);
 }, [queue, isServing]);

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', text: 'Я голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', text: 'Напишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Суп без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">Правило очереди</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 </p>
 </div>
 </div>
 </div>
);

```



```

<div className="grid grid-cols-1 md:grid-cols-12">
```

```
 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
```

```
 <div className={`md:col-span-9 bg-slate-900 rounded-3xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-pulse' : ''}`>
```

```
 <div className="absolute top-4 left-4 text-[18px] font-mono">
 Очередь голодных гостей (Буфер FIFO / BFS)
 </div>
```

```
 <div className="flex-1 flex flex-col justify-between">
 <div className="flex items-end justify-between">
```

```
 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
```

```
 <div className="flex flex-col items-center">
```

```
 <div className="relative">
```

```
 <div className="absolute -top-10 left-1/2 transform">
```

```
 <div className="w-1.5 h-6 bg-slate-400 rounded">
```

```
 <div className="w-2 h-4 bg-slate-400 rounded">
```

```
 </div>
```

```

```

```
 </div>
```

```
 <div className="text-center">
```

```
 <div>
```

```

```

```
 ase font-mono tracking-wider">
```

```
 ПОВАР
```

```

```

```
 </div>
```

```
 </div>
```

```
 {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
```

```
 <div className="flex flex-col items-center">
```

```

```

```

```

```
 </div>
```

```
 {/* СПИСОК ОЧЕРЕДИ */}
```

```
 <div className="flex-1 flex items-end gap-4">
```

```

 `}>
 {c

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[
 ${isHead ? 'bg-amber-500'
 `}>
 {isHead && isTail ? 'h+t'

 })
 </div>
</div>
);
 })
 })
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex i
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 СЫТЫ
 </div>

```

```
 }}}
 </div>
 </div>
);
};
```

---

ФАЙЛ 60/87: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```
export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([]
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>(
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
```

```

 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
}
curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX;
 if (leftX === -1) currentX++;
 }
};

```

```

 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
 });

 // Запуск BFS
 const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
 };

 // Один шаг BFS
 const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);

```

```

<div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-500
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
</div>

```

```

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center">
 <div className="w-40 h-40 relative flex items-center justify-center
 full border border-blue-500/30 shadow-inner">
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_10px_#007bff]' : ''
 }`>
 >
 {/* Тело лосося */}

```

```

 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
 </svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
<span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tra
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leadin
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40
 Статус: {simulationStep === 0 ? 'Бор гото
 оен!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && {

```

```

{(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 /* Draw Edges */
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : ''}
 strokeWidth="2"

```



```

 />
 });
 }}}

 { /* Draw Nodes */
 Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node;
 const isInQueue = bfsQueue.includes(node);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}

```

```

 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-slate-600"
 title="Удалить"
 >
 x
 </button>

)})
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 text-white transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >


```

```

 className={`w-2 h-2 rounded-f
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pi
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }`}
 >
 #{node.id}
 {node.id === 0 && <span className
 </td>
 <td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

 </td>
 <td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
 </td>
 <td className="py-2.5 px-3">
 {node.id === 0 ? (
 <span className="text-slate-5
) : (
 <span className="bg-indigo-95
 #{node.fail}

)}
 </td>
 <td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
) : (
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
 #{node.term_link}


```

```
// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current nodes
 mergeChildren?: [number, number]; // pair of children
 arrayHighlight?: [number, number]; // [L, R]
 result?: number; // partial result
}

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
```

```

// 2. Top-Down Query steps
function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v] });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v] });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v] });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 codeLine: 8, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [l, r] });
 return res;
 }
 const ans = go(1, 0, n - 1, qL, qR);
 steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
 codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
 return steps;
}

```

```

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]

```

```

 return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val: number; }

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m), right: buildVisualTree(treeState, m + 1, r) };
}

// ===== CODE SNIPPETS =====
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {"

```

```

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr: number[] }) {
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree

 const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`}>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-r
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
 </div>
)}
)}
)}

```





```

 setArr(parsed);
 }
};

const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.f
 setArr(a);
 setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSte
const queryBUSteps = useMemo(() => genQueryBottomUpSt

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border bor
 {/* ---- TOP BAR: array editor + query range ----
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4
 {/* Array input */}
 <form onSubmit={handleApply} className="flex-
 <div className="flex items-center justify-b
 <label className="text-[10px] font-bold up
 label>
 <button type="button" onClick={randomize}
 "><RefreshCw className="w-3.5 h-3.5" /></bu
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.'
 className="flex-1 bg-slate-900 border b
 one focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />

```

```

 focus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500" >
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{\text{arr.slice}(q$
 </div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1" >
 <button onClick={() => setView('build')} className="
 -colors ${view === 'build' ? 'border-indigo-500
 -slate-200'}`">
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} class="
 on-colors ${view === 'queries' ? 'border-emerg
 er:text-slate-200'}`">
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class="
 n-colors ${view === 'theory' ? 'border-blue-500
 te-200'}`">
 <Info className="w-3.5 h-3.5" /> Почему / Сра
 </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 >

```

```

 евая часть забирает на 1 больше (округление
 </p>
 <div className="bg-slate-900 p-4 rounded-lg"
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = {(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правы
 h - 1) >> 1)} эл.)</div>
 <div className="text-slate-500 mt-2">Всег
 } внутренних.</div>
 </div>
 </div>

 {/* Comparison cards */}
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <div className="bg-slate-950 p-5 rounded-xl"
 <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 1 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b border-slate-700">
 0(
 <div className="flex justify-between border-b border-slate-700">

 <div className="flex justify-between">✓ легко</div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl"
 <div className="absolute -top-3 left-4 bg-amber-500 border border-amber-500 shadow-md">
 1 Запрос Снизу-вверх (Итерация)
 </div>

```

```
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвигной панели */
 onNavigate?: () => void;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {
 const [query, setQuery] = useState("");

 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 const map = new Map<string, typeof chapters>();
 for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
 }, [query]);

 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:rounded-r-20" style={{ position: "relative" }}>
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute top-0 right-0" />
 <input
 type="text"
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-slate-700 rounded"
 />
 </div>
 </div>
 <div>
 {groups.map(([category, chapters]) => {
 const selectedId = chapters.find((c) => c.id === selectedChapterId)?.id;
 return (
 <div key={category} className="text-slate-400">
 <div>{category}</div>
 <div>
 {chapters.map((c) => {
 const isActive = c.id === selectedId;
 return (
 <div
 key={c.id}
 className={`text-slate-400 hover:text-white ${isActive ? "font-normal" : "font-medium"} cursor-pointer`}
 >
 {c.title}
 </div>
);
 })}
 </div>
 </div>
);
 })}
 </div>
 </aside>
);
};
```

```

 {hasViz && {
 <span
 className="w-1.5 h-1.5 rounded-
 title="Есть интерактивная визуа
 />
 }}
 <span className="font-semibold text
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0
 isSelected ? "text-indigo-400"
 }`
 />
 </button>
);
 }}}
</div>
</div>
)}
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

```

 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"',
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"',
 { id: '3', text: 'Кандидат: "Привет, я испанский ин
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '=' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const poppedPlate = stack[stack.length - 1];
 setStack(stack.slice(0, stack.length - 1));
 setPopMessage(`Убрали тарелку: ${poppedPlate.name}`);
};

```

```

 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (F
 });

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability)
 maxIdx = i;
 }
};

```





```

 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex item
border-blue-500/40 px-4 py-2.5 rounded-xl
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{popMessage && (
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
)}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : (

```

```

 </div>
 </div>
 </div>
 })

 { /* QUEUE TAB */
 {activeTab === 'queue' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-i
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Кто первым пришел, тот первым получил с
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex ite
 -2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex ite
 er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"

```

```

 const isFirst = index === 0;
 return (
 <div
 key={person.id}
 className={`flex flex-col items-center
 ${isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-slate-900
 relative'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 {isFirst && (
 <span className="absolute -top-10 left-1/2
 -60px tracking-wider shadow">
 Первый (Head)

)}
 {person.name}
 <span className={`font-bold text-2xl
 ${isFirst ? 'bg-indigo-500/30 text-white' : ''}
 `}>
 {isFirst ? 'Получает сын' : `В ${person.age} лет`}

 </div>
);
)}
)}

```

```

 <div className="flex flex-col items-center justify-center
 h-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />

 {person.name}

 </div>
 </div>
)}
<div className="mt-6 text-xs text-slate-500">
 {person.name}

```

```
</div>
```

```
{/* Add Form */}
```

```
<form onSubmit={addHypothesis} className="bg-slate-100 p-4 items-end">
```

```
 <div className="md:col-span-6">
```

```
 <label className="block text-xs font-bold">
```

```
 <input
```

```
 type="text"
```

```
 value={newCustomText}
```

```
 onChange={e => setNewCustomText(e.target.value)}
```

```
 placeholder="например: 'Привет, я лучший!'"
```

```
 className="w-full bg-slate-900 border border-slate-500 transition-colors"
```

```
 />
```

```
 />
```

```
 </div>
```

```
 <div className="md:col-span-3">
```

```
 <label className="block text-xs font-bold">
```

```
 Вероятность:
```

```
 </label>
```

```
 <input
```

```
 type="range"
```

```
 min="0.01"
```

```
 max="0.99"
```

```
 step="0.01"
```

```
 value={newCustomProb}
```

```
 onChange={e => setNewCustomProb(Number(e.target.value))}
```

```
 className="w-full accent-emerald-500 cursor-pointer"
```

```
 />
```

```
 </div>
```

```
 <div className="md:col-span-3">
```

```
 <button
```

```
 type="submit"
```

```
 disabled={!newCustomText.trim()}>
```

```
 className="w-full flex items-center justify-center gap-2 text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed h-10"
```

```
 />
```

```
 />
```

```
>
```

```
00/10 scale-[1.01]'
 : 'bg-slate-900/80 border-slate-900'
 }`}
```

```
>
 <div className="flex items-center justify-between"
 <span className={`flex items-center justify-between
 isTop ? 'bg-emerald-500 text-white' : 'bg-white text-slate-900'
 }`}>
 #{index + 1}

 <div>
 <div className={`font-bold text-center`}
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px] font-medium"
 -0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Будущее

)}
 </div>
</div>
```

```
<div className="flex items-center justify-between"
 {/* Custom progress bar */}
 <div className="hidden sm:block"
 <div
 className={`h-full rounded-full`}
 style={{ width: `${percent}%` }}
 ></div>
 </div>
 <span className={`font-mono font-medium`}
 isTop ? 'text-emerald-400 text-white' : 'text-slate-900'
 }`}>
 {percent}%

</div>
</div>
```

```

export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");

 const items = useMemo(() => {
 const titleById = new Map(chapters.map((c) => [c.id, c.title]));
 const all = Object.entries(VIZ_REGISTRY).map(([id, entry], index) => ({
 id,
 entry,
 chapterTitle: titleById.get(id),
 }));
 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q) ||
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
 }, [query]);

 return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">Визуализация
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 Здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-slate-800
 focus:outline-none focus:border-indigo-500"/>
```



```

 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
 },
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }
}

```



```

 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 };
}

```

/\* ----- КОМПОНЕНТ -----

```

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.tree);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; count: number }>({});
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 },
 [presetIdx]
);

```

```

 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium transition-colors shrink-0"
 >
 <Dice className="w-3.5 h-3.5" /> Другое дерево
 </button>
</div>

<div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11px] font-medium transition-colors shrink-0 ${
 i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-600"
 }`}
 >
 {p.name}
 </button>
))}
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);

```

```

 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
 }
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1.5 text-white ${
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 </button>

```

```

 })
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
 })
 </div>
 })
</div>
);
};

```

ФАЙЛ 66/87: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

/\*\*

\* Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag

\*

\* Что сделано ради понятности:

\* \* у узлов настоящие числа (20 / 40 / 60), а не абстрактные

\* сразу видно, что дерево остаётся деревом поиска;

\* \* под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ

\* это и есть доказательство, что поворот ничего не ломает

\* \* кадр «прицел» ничего не двигает, только подсвечивает

\* \* на кадре «результат» остаются призраки – пунктирные

\* места и стрелки «откуда → куда»;

\* \* по умолчанию анимация НЕ крутится сама: пользователь

\* идёт в своём темпе. Автопрокрутка включается кнопкой

\*/

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";

```

```

type Pos = Partial<Record<NodeId, [number, number]>>;

```

```

interface Frame {

```

```

 key: "zig",
 label: "Zig",
 when: "родитель x — уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130] },
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p", "C"]],
 kind: "result",
 title: "Стало: 20 — корень",
 text: "40 стало правым сыном двадцатки. Поддеревом",
 text2: "ева от 40 — это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ S0 = {

```

```

 text: "40 встало в корень, 60 опустилось к нему п
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {

```

```

 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

```

```

const COLOR: Record<string, { fill: string; stroke: str
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x – ищем его",
 p: "p – родитель",
 g: "g – дед",
};

```

```

const isSubtree = (id: NodeId) => id === "A" || id ===

```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return {

```

```

 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a) && focus.includes(b));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.16, 1, 0.3, 0.2)" }}
 />
);
 }
}
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 />
 </g>
);
}
}

```



```
const t = setTimeout(() => setFrame((f) => (f + 1) * 1000), 1000);
return () => clearTimeout(t);
}, [playing, frame, duration, total]);
```

```
return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-white
 i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800"
 }`}
 >
 {c.label}
 </button>
))}
 </div>
```

```
<div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корреляция
 ({active.rotation}
</div>
```

/\* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение

```
<div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`}
 >
 {current.kind === "aim" ? "прицел" : current.kind}

 </div>
```

```

 ? { animation: `demoProgress ${duration}ms`
 : { width: "100%", background: "#334155"
 }
 />
</div>

<div className="flex flex-wrap items-center gap-2" >
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800 text-white
 dark:hover:text-slate-300 text-xs font-bold transition-colors"
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600 text-white
 w-indigo-900/40"
 >
 {last ? "в Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2 text-white
 text-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" /> : "Автомат"}
 {playing ? "Стоп" : "Автомат"}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}

```

---

ФАЙЛ 67/87: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import {
 RotateCcw,
 HelpCircle,
 BookOpen,
} from "lucide-react";

interface SplayNode {
 key: number;
 left: SplayNode | null;
 right: SplayNode | null;
 x?: number;
 y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number) => {
 if (!root) return { key, left: null, right: null };
 if (key < root.key) {
 root.left = insertBST(root.left, key);
 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
```

```

const [log, setLog] = useState<string[]>([
 "Дерево инициализировано. Попробуйте кликнуть на узел",
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
 root: SplayNode | null,
 key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
 const steps: string[] = [];
 if (!root) return { newRoot: null, steps: ["Дерево пустое"] };

 // We implement the classic top-down or bottom-up splay operation.
 // For visualization logs, we do a bottom-up explanation.
 let path: { node: SplayNode; dir: "left" | "right" } | null = null;
 let current: SplayNode | null = root;

 // Find the element or the element where search failed.
 let lastNode: SplayNode = root;
 while (current) {
 lastNode = current;
 if (key === current.key) {
 path.push({ node: current, dir: null });
 break;
 } else if (key < current.key) {
 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
 }

 const targetKey = current ? key : lastNode.key;
 const isFound = !!current;

 if (!isFound) {

```

```

 `Финальный поворот Zig (Правое вращение корня
);
 return rightRotate(node);
} else {
 if (!node.right) return node;

 if (k < node.right.key) {
 // Zag-Zig (Right-Left)
 node.right.left = splayAction(node.right.left, target);
 if (node.right.left) {
 node.right = rightRotate(node.right);
 steps.push(`Компонент Zig-Zag: правый поворот
);
 } else if (k > node.right.key) {
 // Zag-Zag (Right-Right)
 node.right.right = splayAction(node.right.right, target);
 node = leftRotate(node);
 steps.push(
 `Вращение Zag-Zag (Левый поворот родителя д
);
 }

 if (!node.right) return node;
 steps.push(
 `Финальный поворот Zag (Левое вращение корня
);
 return leftRotate(node);
}
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {

```

```

const computeCoordinates = (
 node: SplayNode | null,
 x: number,
 y: number,
 levelWidth: number,
): void => {
 if (!node) return;
 node.x = x;
 node.y = y;
 if (node.left) {
 computeCoordinates(node.left, x - levelWidth, y +
 }
 if (node.right) {
 computeCoordinates(node.right, x + levelWidth, y
 }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.R
 if (!node) return [];
 let lines: React.ReactNode[] = [];
 if (node.left && node.left.x && node.left.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
}

```

```

 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="12px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.key}
 </text>
 </g>,
);
 circles = circles.concat(renderNodes(node.left));
 circles = circles.concat(renderNodes(node.right));
 return circles;
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 { /* Header */ }
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <BookOpen className="w-5 h-5 text-indigo-400"
 Интерактивное Splay-дерево
 </h3>
 <p className="text-xs text-slate-400">
 Кликните на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 { /* Playfield */ }
 <div className="lg:col-span-7 bg-[#0f172a] p-4
 <svg
 className="w-full h-full max-w-[600px] max-h-

```

```

 }}
 className="bg-indigo-950 hover:bg-slate
0"
 >
 Вставить
 </button>
</form>

<button
 onClick={resetTree}
 className="flex items-center gap-1.5 px-3
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>
</div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border
 to overflow-y-auto custom-scrollbar">
 <h4 className="text-xs font-bold text-slate-400">
 Ход вращения (Splay-логи)
 </h4>
 <div className="flex-1 space-y-2 mb-4 overflow
 {log.map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2.5 rounded trans
 idx === log.length - 1
 ? "border-l-2 border-indigo-500 bg-
 : "text-slate-400 bg-slate-900/50"
 }`}
 >
 {step}
 </div>
 })}
 </div>

```



```

 <strong className="text-white">
 последний успешно просмотренный узел

 , на котором он споткнулся! Это оставля
 сверху.
 </p>
 </div>
 <div className="mt-3 text-[10px] text-indigo
 Поиск настраивает локальный кэш под реали
 </div>
 </div>

<div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb
 2. Эффект качелей (Swing)
 </p>
 <p className="text-slate-400 leading-rela
 Если агент запрашивает по очереди{" "}
 <code className="text-rose-400">[10]</code>
 <code className="text-rose-400">[60]</code>
 углы), дерево будет превращаться в палк
 сторону. Первая операция Splay(60) стои
 <strong className="text-white">0(n)</st
 глубину n. Вторая Splay(10) заставляет
 Постоянный свинг убивает производительн
 сложности <strong className="text-white
 </p>
 </div>
 <div className="mt-3 text-[10px] text-rose-
 Кэшировать свингующие пары на концах – фа
 </div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb
 3. Амортизация $O(\log n)$

```

```

}

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'f' },
 { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'f' },
 { name: 'Блюдец от торта', emoji: ' ', color: 'from-p' },
 { name: 'Тарелка от тако', emoji: ' ', color: 'from-y' },
 { name: 'Тарелка из-под суши', emoji: ' ', color: 'fr' },
 { name: 'Сковорода от глазуньи', emoji: ' ', color: '' },
];

let counter = 0;

export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>(
 [
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', color: 'f', state: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', color: 'f', state: 'idle' },
 { id: 'p3', name: 'Блюдец от торта', emoji: ' ', color: 'from-p', state: 'idle' },
]
);

 const [cleanRack, setCleanRack] = useState<Plate[]>(
 []
);
 const [isWashing, setIsWashing] = useState(false);
 const [showSoap, setShowSoap] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>(
 []
);

 const addLog = (msg: string) => setLog(prev => [...prev, msg]);

 // PUSH: Положить грязную тарелку сверху стопки
 const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 }
 }, [isWashing]);

```

```

 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(` Готово! Чистая тарелка ${topPlate.emoji}`, 850);
 }, [dirtyStack, isWashing]);

const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};

const topIndex = dirtyStack.length - 1;

return (
 <div className="flex flex-col gap-6">

```

```

 r-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
 <div className="md:col-span-7 bg-slate-900 rounded-3xl
 [380px]">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 {/* Визуализация раковины */}
 <div className="absolute top-4 right-4 flex items-center gap-2" >
 r-slate-800/80 text-[10px] font-mono">

 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && {
 <div className="absolute inset-0 pointer-events-none" >
 <div className="absolute w-6 h-6 rounded-full bg-white
 " ></div>
 <div className="absolute w-4 h-4 rounded-full bg-white
 " ></div>
 <div className="absolute w-5 h-5 rounded-full bg-white
 " ></div>
 <div className="absolute text-4xl anim-crash" >
 </div>
 </div>
 }}

 {dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center" >

```

```

 {isTop && (
 <span className="text-[9px] bg-blur-2xl
 LIFO

)}
 </div>
);
 }
 </div>
)}

 {/* Столешница раковины */}
 <div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-800" />
</div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-3xl p-4" />
 <div className="absolute top-4 left-4 text-[10px] font-mono" />
 Сушилка для чистой посуды
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" />
 <div className="order-emerald-800/80 text-[9px] font-mono" />
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
 </div>

 <div className="flex-1 flex flex-col justify-between p-4" />
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2" />
 Пусто
 <p className="text-xs font-bold font-mono">Здесь можно разместить тарелки</p>
 <p className="text-[10px] mt-1">Здесь 6 тарелок</p>
 </div>
) : (
 <div className="flex flex-col gap-2 w-full" />
 {cleanRack.map(plate => (
 <div

```

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];

 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадения нет` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записываем` });
 }
}
```

```

 [i]]}' разные. Стоп.` }));
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упёр` });
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксирыв` });

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: `0` });
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true });
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

```

```

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

```

```

 const { s, steps, patternLength } = useMemo(() => {
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);

```

```

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 useVizStepSync(stepIdx, setStepIdx, steps.length - 1);
 const timer = useRef<NodeJS.Timeout | null>(null);

```

```

 " : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ

```

```

 </button>

```

```

 </div>

```

```

 <div className="flex items-center gap-2"

```

```

 <div className="bg-slate-800 p-1 fl

```

```

 <span className="text-[10px] te

```

```

 <input value={pattern} onChange=

```

```

xt-white text-xs font-bold font-mono px-2 p

```

```

 </div>

```

```

 <div className="bg-slate-800 p-1 fl

```

```

 <span className="text-[10px] te

```

```

 <input value={text} onChange={e

```

```

te text-xs font-bold font-mono px-2 py-1 ou

```

```

 </div>

```

```

 </div>

```

```

</div>

```

```

<div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">

```

```

 <div className="flex gap-1.5 px-4 h-ful

```

```

 {s.split("").map((char, index) => {

```

```

 const isPattern = index < patte

```

```

 const isSeparator = index === p

```

```

 let borderColor = "border-slate

```

```

 let bgColor = "bg-slate-900";

```

```

 let textColor = isSeparator ? "

```

```

 // KMP Logic

```

```

 if (mode === "kmp") {

```

```

 if (step.highlight?.include

```

```

 borderColor = "border-a

```

```

 bgColor = step.match ===

```

```

-900/40");

```

```

 }

```

```

 }

```



```

z-20">
 { /* KMP fingers */
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}
 {mode == "kmp" && step
 <div className="abs
 <span className
 </div>
 }}

20">
 { /* Z fingers */
 {mode == "z" && step.i
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}

e z-20">
 { /* Z fingers */
 {mode == "z" && step.i
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}

```

```

 overflow-hidden">
 <div className="absolute top-0 right-0" >

 {steps.length}
 </div>
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>

 { /* Why this matters card */ }
 <div className="bg-slate-800 p-4 rounded-xl">

 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "проблема не была найдена!" : "проблема найдена!"}
 Пройди шаги вручную, посмотри, как работает алгоритм для пропуска работы (в Z).
 </p>
 </div>

 { /* Code Sneak Peek */ }
 <div className="bg-slate-900 border border-slate-800 px-4 py-2">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase">
 Код C++ {mode === 'kmp' ? 'КМП' : 'ЗКМП'}
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400">
{mode === 'kmp' ? `vector<int> pi(s.length());` : `vector<int> pi(s.length());`}
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
 </pre>
 </div>
 </div>

```

```

const [open, setOpen] = useState(false);

const hiddenShift = side === "top" ? "translate-y-1"

return (
 <span
 className={`relative inline-flex ${className} ?? ""
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1
 bg-slate-900 px-3 py-2 text-left text-[11px]
 l duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full
 } ${open ? "visible translate-y-0 opacity-100"
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
};

```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number | null) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);
 });
};
```

```

 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
}
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {

```



```

 textcolor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textcolor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-!
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"

```

```

 </div>

 { /* Color key */ }
 <div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 >
 Далее
 </button>
 </div>
 </div>
</div>

```



## Универсальное пособие • полный исходный код

```

import { SegmentTreeVisualizer } from "../SegmentTreeViz";
import { SplayTreeViz } from "../SplayTreeViz";
import { SplayRotationsViz } from "../SplayRotationsViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithmsViz";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimation";
import ChapterImage from "../ChapterImage";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTableViz";

/** Разбор поворота и песочница всегда идут парой: посмотри! */
/**
 * Анимация и песочница идут друг под другом, а не в две
 * колонке дерево сжималось до нечитаемого размера, а по
 * компактности. Между ними – подпись, объясняющая перемену
 */
const SplayRotationsPair: React.FC = () => (
 <div className="space-y-4">
 <SplayRotationsViz />
 <p className="flex items-start gap-2 text-[12px] leading-3 py-2">

 Разобрался – проверь себя: ниже то же самое дерево
 не решишь.

 </p>
 <SplayRotationSandbox />
 </div>
);

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 description: string;
}
```

```

 Component: SparseTableViz,
 },
 "prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сложилось",
 Component: PrefixSumViz,
 },
 "dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
 },
 "splay-tree": {
 title: "Splay-дерево",
 hint: "Покадровый разбор трёх поворотов, песочница",
 Component: () => (
 <div className="space-y-10">
 <SplayRotationsPair />
 <SplayTreeViz />
 </div>
),
 },
 "splay-rotations": {
 title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
 hint: "Сверху — покадровый разбор поворота, снизу — результат",
 Component: () => <SplayRotationsPair />,
 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDFS },
 "top-sort": { title: "Топологическая сортировка", Component: TopSort },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC },
 "graph-articulation": { title: "Точки сочленения", Component: Articulation },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: Bridges },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: Euler },
 "planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: Planarity },
 "dijkstra": {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (

```

```
 },
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },
};

export const getViz = (id?: string): VizEntry | undefined;
```

---

ФАЙЛ 73/87: src/components/WaterfallAnimationWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
```

```

const [searchLog, setSearchLog] = useState<string>(
 'Режим поиска: Введите текст и жмите «Следующий шаг»
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({
 from: 0, to: 0
});
const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

```

```

 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentIndex, word: word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog[0].word} `;
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMatches]);
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Остальные вершины сразу инициализируются с fail = ROOT',
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди'
 }
];

```

```

 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ ШЕЛКАМ?',
 desc: 'ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2. Рыба плывет в ROOT и ищет ветку "H". У корня суффикс "H"! ',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель #3 (HER) не подходит. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель #6 (HIS) не подходит. Но ветка "S" → #7 (S) совпадает! fail[HIS] = #7 (S)',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,

```

```

 </p>
 </div>
 </div>

 { /* Переключатель режимов */ }
 <div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg sl
 : 'text-slate-400 hover:text-white'
 }` }
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 }` }
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
 </div>

 { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
 { activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 { /* Управление шагами обучения */ }
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>

```

```

 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.length
 </button>
</div>
</div>

```

```

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
 >

```

```

 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>

```

```

 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>

```

```

 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>

```

```

 <div className={`p-1.5 rounded flex ite
-l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <

```



```

 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-b
cus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1"
 <span className="text-xs text-slate-400"
 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center
 idx === currentIndex
 ? 'bg-blue-600 text-white shadow
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emera
 : 'bg-slate-800 text-slate-500'
 }`}
 >
 {char}

)}
 </div>
 {currentIndex >= textToScan.length && (
 <span className="bg-emerald-600 text-
 ГОТОВО ✓

)}
 </div>

 <button

```

```

 </div>
 <div className={`p-1.5 rounded flex items-center justify-between border-emerald-500 text-emerald-300 font-mono`>
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 &&
 совпадение!}
 </div>
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-emerald-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-md flex items-center">
 {searchLog}
 </div>
</div>
</div>
</div>
))

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
 <div className="absolute inset-0 opacity-10 bg-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-50% w-8 bg-blue-400 h-full animate-fall">
 <div className="w-12 bg-blue-300 h-full animate-fall">
 <div className="w-6 bg-blue-500 h-full animate-fall">
 </div>
 </div>
 </div>

 <div className="flex items-center justify-between p-1.5">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Бо

```



```

const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2);
const cy = node.y + dy / 2 - 30;

return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'tracking' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

return (
 <g key={`node-${node.id}`} className={isCurrent ? 'active' : ''}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r="100" fill="none" stroke="red" stroke-width="2" />
)}
 <circle cx={node.x} cy={node.y} r="100" fill={node.fill} stroke={node.stroke} stroke-width="2" />
 </g>
);

```

```

 {hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
);
}}
}}}

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
<g>
 {/* Круг-подложка под рыбу */
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 {/* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
</g>

```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 74/87: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-monospace" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border" />
);
}
```

```

 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'range' })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r ? r : i))}
 className={` ${cellBase} cursor-pointer`}>
 {v}
 {
 inCover
 ? "bg-indigo-500 text-white ring-1 ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }
 </div>
 </div>
)
)}
 </div>
 <div>
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'value' })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-pointer`}>
 {v}
 {
 inCover
 ? "bg-indigo-500 text-white ring-1 ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }
 </div>
 </div>
)
)}
 </div>
 </div>
 </div>
 </div>
 </div>
)

```

```

 { /* Пояснение под курсором */ }
 <div className="min-h-[62px] bg-slate-900 border border-indigo-500" >
 { hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0 — начало
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 — это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-emerald-400">у
 у <b className="text-indigo-400">a — к у,
 </p>
)
 }
 </div>

```

```

 { /* Запрос */ }
 <div className="bg-slate-900 border border-indigo-500" >

```



```

const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
);
 })}
 </div>
)};
 </div>
 </div>
 </div>
 </div>
);
}
}
}

```

```

 </div>
);
 }}}
</div>
 }}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j]\}$
 { " " }
 – сумма всего прямоугольника от левого верхнего угла
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-500">
 сумма = A + <span
 C + <span
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};

```

---

```

];

const ST2D: number[][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array(4).fill(0)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

export const SparseTableViz: React.FC = () => {

```



```

 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\text{hover.j}}$ = {1 << hover.j}:{ " "}

 min({arrID.slice(covered.from, covered.to)})

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-50">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-sm">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-sm">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-sm">
 2^2 (L=4)
 </div>
 </div>
 </div>
</div>

```

```

 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 =
 if (currentStep.i + (1 << (currentStep.j - 1)) === i) {
 isSrc2 = true;
 }
 }
 }
 }

 return (
 <div key={j} className="relative flex flex-col items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200"
 > {
 <motion.div
 onMouseEnter={() => isVisible}
 onMouseLeave={() => setHover(false)}
 onClick={() => isVisible && setHover(true)}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200
 hover && hover.i === i && hover.s === s
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"}
 > {st1D[i][j]}
 </motion.div>
 </div>
) : (
 <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200"
 > {
 <motion.div
 onMouseEnter={() => isVisible}
 onMouseLeave={() => setHover(false)}
 onClick={() => isVisible && setHover(true)}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200
 hover && hover.i === i && hover.s === s
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"}
 > {st1D[i][j]}
 </motion.div>
 </div>
)}
)
)
}

```

```

 const half = 1 << (c.j - 1);
 return {
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-b
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);
 })()
) : {
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);

```

```

<React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft

```



```

 </div>
 </div>

 <div className="flex-1 min-w-0 flex flex-col gap-1" >
 <div className="bg-slate-900 p-4 sm:p-6 rounded" >
 <h3 className="text-lg font-bold text-slate-300" >
 {k === 0 ? (
 <div className="text-slate-400 text-sm leading-4" >
 <p className="mb-4" >При
 и имеют размер
 <div className="bg-[#0a0f1e] rounded-xl p-2" >
 // Ба
 ST
 </div>
 <p className="mt-6 italic text-indigo-400" >
 <Play size={14} className="fill-indigo-400" >
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-fade-in" >
 <div className="bg-[#0a0f1e] rounded-xl p-2 text-slate-300 shadow-inner overflow-x-auto" >
 // Те
 int1});

 // Ква
 ST
 1],
 1],
 1],
 1][k-

 1],
 1][k-

 1],
 c + pL][k-

 1],
 // Правый Низ


```

```

 >{ST2D[activeCell.r + prevL][activeCell.c][k]}
 </div>
 <div className="flex item-center justify-center" style={
 shadow-[0_0_8px_#f59e0b]}></div> ST[{activeCell.r + prevL}][
 {ST2D[activeCell.r + prevL][activeCell.c + prevL][k]}
 </div>
 </>
)
 }}()
</div>

 <div className="pt-3 mt-3 border-top border-left border-right border-bottom" style={
 }> {ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font-mono">
 список матриц (слева) вниз, чтобы увидеть, как они меняются.
 </p>
) : {
 <div className="bg-slate-950/40 border border-slate-700 p-2 flex items-center justify-center animate-pulse" style={
 }>
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
</div>
)}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);

```

```

ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

```

```

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-events-none" style={{
 shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
 <div className="absolute pointer-events-none" style={{
 shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
 width: `calc(
 <div className="absolute pointer-events-none" style={{
 ld-400 shadow-[0_0_10px_#10b981]" style={{ top: `calc(
 8px)` , width: `calc(
 <div className="absolute pointer-events-none" style={{
 00 shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
 00)% + 8px)` , width: `calc(
 </div>

```

```

 <div className="bg-slate-950 p-4 rounded-lg" style={{
 er-slate-800 shadow-[inset_0_2px_10px_rgba(0,0,0,0.3)]" style={{
 <label className="flex items-center justify-between" style={{
 <span className="text-slate-400" style={{
 <div className="flex gap-2" style={{
 <input type="number" min={0} max={100} value={0} style={{
 14 bg-slate-900 border border-slate-700 py-2 px-4" style={{
 <input type="number" min={0} max={100} value={0} style={{
 14 bg-slate-900 border border-slate-700 py-2 px-4" style={{
 </div>
 </label>
 <label className="flex items-center justify-between" style={{

```



```

 <span className="text-amber-400"
 <span className="text-slate-400"
 <span className="ml-3 text-emerald-400"
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly]
 </div>
 </div>
 </div>
)
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

ФАЙЛ 76/87: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРӨК: 137 | РАЗМЕР

```

 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').cla
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 document.getElementById('btn-mode-wtf').cla
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold'

 const aside = document.querySelector('aside
```

```

 document.getElementById('btn-theme-dark').classList.remove('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlighting
 document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('hidden');
 if (isClosed) {
 mobileDrawer.classList.remove('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = "";
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
 });

```

```
 }
 });
 }
 });
}, observerOptions);

document.querySelectorAll('section[id^="que
 observer.observe(section);
});
});
</script>
</body>
</html>
```

---

ФАЙЛ 77/87: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 1169 | РАЗМЕР: 1000

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.1/es5/tex-mml-chtml.js"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 },
 };
 </script>
 <style>
```

```

 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px;

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::after

 .uno-equation .text-2xl { font-size: 1.2rem
 .uno-equation.gap-4 { gap: 0.5rem !important

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%
 border-radius: 50%; transform: rotate(-30deg
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card
 .card-green { background-color: #339e35; } .card
 .card-yellow { background-color: #ffc400; } .ca

```



```
 background-color: rgba(255, 255, 255, 0.7)
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
 }
 body.theme-notebook .text-slate-200, body.theme
 ant; }
 body.theme-notebook .text-slate-400 { color: #5
 body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
 body.theme-notebook .font-mono { font-family: "
 body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
```

```

</marker>
<marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf
</marker>
<marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63
</marker>
<marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->

```

```
 id="text-sm">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">

 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>
 </nav>
 </div>
</div>
```

```

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q5-def" class="block l
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
```



```
deg)
 <a href="#q14-4" class="block
ition-colors border border-rose-500/30"> K
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2">
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-16" clas
der-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч

 </summary>
 <div class="pl-6 mt-2 space-y-2">
 <a href="#q16-1" class="blo
 <a href="#q16-2" class="blo
 <a href="#q16-3" class="blo
 </div>
</details>

<div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
```

```

 }
 });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 "#q5", div[id^="q6-"], div[id^="q7-"], div[id^="q8-"],
 div[id^="q9-"], div[id^="q10-"], div[id^="q11-"], div[id^="q12-"],
 div[id^="q13-"], div[id^="q14-"], div[id^="q15-"]
);
 let isScrolling = false;

 document.querySelector('.global-nav').addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 200);
 });

 const observer = new IntersectionObserver((entries) => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('.global-nav a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
 if (correspondingLink) {
 const details = document.querySelector(`details#${visibleId}`);
 details.open = true;
 }
 });
});

```

```
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за не
ние веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
```



```

// Smart markdown parsing based

// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
});
clone.querySelectorAll('.img-cap

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_algel
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate

```

```

 body.bg-white .analogy-l
or: #94a3b8 !important; }
 body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`
} else if (theme === 'terminal'
 body.classList.add('bg-blac
 document.getElementById('th

```

```

 -bold flex items-center justify-center gap-2
 Markdown
 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
 </div>
 </div>
 </div>
</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bold">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">

```

```

 <div class="bg-yellow-900/40 border-4 border-yellow-500 hover:bg-slate-700 transition" data-bbox="273 101 1000 146">
 оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 border-4 border-orange-500 hover:bg-slate-700 transition" data-bbox="273 151 1000 196">
 олько полиномов</div>
 <div class="bg-cyan-900/40 border-4 border-cyan-500 hover:bg-slate-700 transition" data-bbox="273 201 1000 246">
 сные числа</div>

 <!-- Row 1: База -->

 <div class="text-xs text-yellow" data-bbox="473 341 1000 361">
 <div class="font-bold text-slate-700" data-bbox="473 366 1000 386">
 </div>

 <div class="text-xs text-orange" data-bbox="473 481 1000 501">
 <div class="font-bold text-slate-700" data-bbox="473 506 1000 526">
 </div>

 <div class="text-xs text-cyan-500" data-bbox="473 621 1000 641">
 <div class="font-bold text-slate-700" data-bbox="473 646 1000 666">
 </div>

 <!-- Row 2: Базовые операции / Евклид -->

 <div class="text-xs text-yellow" data-bbox="473 761 1000 781">
 <div class="font-bold text-slate-700" data-bbox="473 786 1000 806">
 </div>

 <div class="text-xs text-orange" data-bbox="473 901 1000 921">
 <div class="font-bold text-slate-700" data-bbox="473 926 1000 946">
 </div>

 <div class="text-xs text-cyan-500" data-bbox="473 1001 1000 1000">

```

```

 <div class="text-xs text-orange">
 <div class="font-bold text-slate-700">

-4 border-cyan-500 hover:bg-slate-700 transition-colors duration-200"
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200"
 <div class="text-xs text-orange">
 <div class="font-bold text-slate-700">

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200"
 <div class="text-xs text-orange">
 <div class="font-bold text-slate-700">

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-3">
b-2 border-b border-emerald-500/30 pb-2">Кл

-1-4 border-teal-500 hover:bg-slate-700 transition-colors duration-200"
 <div class="text-xs text-teal-500">
 <div class="font-bold text-slate-700">

-1-4 border-blue-500 hover:bg-slate-700 transition-colors duration-200"

```

```
<section id="proof-algorithms" class="m
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 r
 <svg class="w-8 h-8 text-re
rg/2000/svg">
 <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border
 </div>

 <div class="prose prose-invert max-w
 <p class="text-lg">
 Хватит тупить. Доказательств
есть для того, чтобы жать на кнопки. Вот жес
 </p>

 <h2 class="text-2xl font-bold b
(Меметика)</h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 s
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

 </div>

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-fuch
 <p class="text-sm text-slate
```

```
<div class="bg-slate-800 p-6">
 <h3 class="text-xl text-orange-400">
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5">

 вверх по уравнениям).

 рху вниз, показывая, что $\Delta \vdots r_i$

 х. Твой d больше.

 </div>
```

```
<h2 class="text-2xl font-bold text-white">
 ритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2">
 <div class="bg-slate-800 p-6">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-sans">
 со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
<div class="bg-slate-800 p-6">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-sans">
 c . Пока получается 0 – корень жив. Как то
 корня минус один.</p>
 </div>
```

```
<div class="bg-slate-800 p-6">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-sans">
 циентов (они делятся на p), кроме старше
```

```
 <li class="bg-slate-800/50" >

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение — есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50" >

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

 </div>
 </section>
 </div>

 <div id="questions-container">
 <!-- ===== ВОПРОС 1 =====
```

---

РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 78/87: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---



```

 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
}
return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important
 pre { white-space: pre-wrap; }
}
`;

```

```

const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(id)}">
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
 </div>`;
 })
 .join("\n");

return `<section class="export-gloss">
 <h2>Словарик терминов из этой темы</h2>
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
 ${items}
</section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; }) {
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">
 <div style="font-size:.72rem;color:#475569;margin-top:10px">

```



```

const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")]
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^\\d+\\./)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.

```

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\
const re = new RegExp(
 `(?!<![\\p{L}\\p{N}_-])(<${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
```

```
* ШАГ 3: PNG -> PDF
*
* Склеивает отрендеренные страницы tmp/export-pages/pages/
* в единый документ public/export/full_code_all_pages.pdf
*/
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });
 doc.pipe(fs.createWriteStream(PDF_PATH));
 doc.end();
}
```

## Универсальное пособие • полный исходный код

-----  
\* с фоллбэком на обход файловой системы, если git недо-  
\*/

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```

```
/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);
```

```
/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",
```

```
/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в.
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+\$/ , "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel))
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
```

```

 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
 }

 /** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^\"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti

```



```
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`файлов: ${files.length}`);
console.log(`глав: ${chapters.length}`);
console.log(`строк: ${txt.split("\n").length}`);
console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();
```

```

 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /** пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
);
}

```

```

const source = txt.replace(/\r\n/g, "\n").split("\n");
const pages = [];
for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
}
return pages;
}

```

```

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

```

```

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

```

```

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

```

```

const total = pages.length;
const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
});

```

```

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
];

```

```

 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
 });
```

=====

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

-----

ФАЙЛ 86/87: .github/workflows/deploy-pages.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9

-----

name: Deploy app to GitHub Pages

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
(у бота-интеграции нет admin-прав на них – API отвечает)
#
1. Settings → Pages → Build and deployment → Source
Если там «Deploy from a branch» (legacy) – шаг D
2. Settings → Environments → github-pages → Deployment
добавьте паттерн «arena/**» и ветку «main» (или
иначе деплой из новой рабочей ветки отклоняется)
```

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

permissions:

contents: read

pages: write

id-token: write

```
environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
steps:
 # Ранняя и понятная ошибка вместо невнятного «Fail»
 # когда в репозитории Pages не переключён на сбор
 - name: Preflight – Pages собирается через GitHub
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages/build_types" |
 echo "Pages build_type: ${bt}"
 if ["${bt}" != "workflow"]; then
 echo "::error title=Pages не настроен::Settings не настроены. См. комментарий в начале деплоя"
 exit 1
 fi

 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

---

ФАЙЛ 87/87: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

---

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

- name: Setup Node.js  
uses: actions/setup-node@v7  
with:  
  node-version: "24"  
  cache: npm
- name: Install ImageMagick + DejaVu fonts  
run: |  
  sudo apt-get update -qq  
  sudo apt-get install -y --no-install-recommends  
  # На Ubuntu политика ImageMagick иногда запрещает экспорт в pdf  
  # разрешаем то, что нужно пайплайну (text -> pdf)  
  for policy in /etc/ImageMagick-6/policy.xml /usr/lib/policy.xml; do  
    if [ -f "\$policy" ]; then  
      sudo sed -i 's/rights="none" pattern="\{PDF,PSD\}" rights="write,delete" /rights="write,delete" /g'  
    fi  
  done  
  convert -version
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.density:-300}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true

```
fi
git commit -m "chore(export): пересборка full
git push origin "HEAD:${GITHUB_REF_NAME}"
```